

读享 V1.0 软件

后 30 页

著作权人:XX 有限公司

生成日期:2026-06-15

摘选总行数:5536

文件清单:

- person/notfresh/readingshare/core/Synchronizer.java
- person/notfresh/readingshare/util/PdfOutlineExtractor.java
- person/notfresh/readingshare/util/BilibiliUrlConverter.java
- person/notfresh/readingshare/util/CrawlUtil.java
- person/notfresh/readingshare/util/ExportUtil.java
- person/notfresh/readingshare/util/ImportUtil.java
- person/notfresh/readingshare/util/ImageUtil.java
- person/notfresh/readingshare/util/SubjectUtil.java
- person/notfresh/readingshare/util/SwipeActionsHelper.java
- person/notfresh/readingshare/ui/subject/SubjectFragment.java
- person/notfresh/readingshare/ui/document/DocumentViewerActivity.java
- person/notfresh/readingshare/WebViewActivity.java
- person/notfresh/readingshare/WebViewManager.java
- person/notfresh/readingshare/ClickStatisticsActivity.java
- person/notfresh/readingshare/UserProfileActivity.java
- person/notfresh/readingshare/WebViewBackgroundService.java
- person/notfresh/readingshare/config/Config.java

第2页 / 共112页

100

```

101         linkObj.put("title", link.getTitle());
102         linkObj.put("url", link.getUrl());
103         linkObj.put("timestamp", isoFormat.format(new Date(link.getTimestamp
104         linksArray.put(linkObj);
105     }
106
107     jsonData.put("links", linksArray);
108     return jsonData;
109 }
110
111 /**
112  * ■■■HTTP■■■
113  */
114 private SyncResult sendRequest(String targetUrl, JSONObject jsonData) {
115     try {
116         URL url = new URL(targetUrl);
117         HttpURLConnection conn = (HttpURLConnection) url.openConnection();
118
119         // TODO ■■■SSL■■■■■■■■■■
120         //configureSSL(conn);
121
122         // ■■■■■■■■■■
123         conn.setRequestMethod("POST");
124         conn.setRequestProperty("Content-Type", "application/json");
125         conn.setDoOutput(true);
126
127         Log.d(TAG, "■■■■■■■■: " + targetUrl);
128
129         // ■■■■■■
130         try (OutputStream os = conn.getOutputStream()) {
131             byte[] input = jsonData.toString().getBytes(StandardCharsets.UTF
132             os.write(input, 0, input.length);
133         }
134
135         // ■■■■■■
136         int responseCode = conn.getResponseCode();
137         Log.d(TAG, "■■■■■■■■: " + responseCode);
138
139         if (responseCode == HttpURLConnection.HTTP_OK) {
140             // ■■■■■■
141             String responseData = readResponse(conn);
142             return SyncResult.success(responseData, responseCode);
143         } else {
144             // ■■■■■■
145             String errorResponse = readErrorResponse(conn);
146             return SyncResult.failure("■■■■■■■■: " + responseCode +
147                 (errorResponse != null ? " - " + errorResponse : ""), respon
148         }
149     } catch (IOException e) {

```

第5页 / 共112页

```

201     }
202 }
203
204 /**
205  * ████████
206  */
207 private String readErrorResponse(HttpURLConnection conn) {
208     try {
209         if (conn.getErrorStream() != null) {
210             try (BufferedReader reader = new BufferedReader(
211                 new InputStreamReader(conn.getErrorStream(), StandardCha
212                     StringBuilder response = new StringBuilder();
213                     String line;
214                     while ((line = reader.readLine()) != null) {
215                         response.append(line);
216                     }
217                     return response.toString();
218                 }
219             }
220         } catch (IOException e) {
221             Log.e(TAG, "██████████", e);
222         }
223         return null;
224     }
225 }
226
227 1 package person.notfresh.readingshare.util;
228 2
229 3 import android.util.Log;
230 4
231 5 import java.io.File;
232 6 import java.io.FileInputStream;
233 7 import java.io.IOException;
234 8 import java.nio.charset.StandardCharsets;
235 9 import java.util.ArrayList;
236 10 import java.util.List;
237 11 import java.util.regex.Matcher;
238 12 import java.util.regex.Pattern;
239 13
240 14 import person.notfresh.readingshare.ui.document.TocOutlineItem;
241 15
242 16 /**
243 17  * PDF██████
244 18  * █████PDF██████████/████
245 19  */
246 20 public class PdfOutlineExtractor {
247 21     private static final String TAG = "PdfOutlineExtractor";
248 22
249 23     /**
250 24     * █████PDF██████████

```

第7页 / 共112页

```

75         "<[>]*?/Title\\s*\\(([^\\s]+)\\)[>]*?/Dest\\s*\\([\\s\\S]+)\\)",
76         Pattern.CASE_INSENSITIVE | Pattern.DOTALL
77     );
78
79     Matcher bookmarkMatcher = bookmarkPattern.matcher(pdfContent);
80     int count = 0;
81
82     while (bookmarkMatcher.find() && count < 200) { // 200
83         try {
84             String title = bookmarkMatcher.group(1);
85             String dest = bookmarkMatcher.group(2);
86
87             //
88             int pageIndex = extractPageIndexFromDest(dest);
89
90             if (pageIndex >= 0) {
91                 //
92                 title = cleanTitle(title);
93
94                 //
95                 if (isValidOutlineTitle(title)) {
96                     // Parent/First/Last
97                     int level = estimateLevel(bookmarkMatcher.start(), p
98
99                     TocOutlineItem item = new TocOutlineItem(title, page
100                     outlineItems.add(item);
101                     count++;
102
103                     Log.d(TAG, " : " + title + " -> " + (pageIndex
104                 }
105             }
106         } catch (Exception e) {
107             Log.w(TAG, "", e);
108         }
109     }
110
111     // 21 /Title
112     if (outlineItems.isEmpty()) {
113         extractOutlinesSimple(pdfContent, outlineItems);
114     }
115
116     //
117     deduplicateAndSort(outlineItems);
118
119     } catch (Exception e) {
120         Log.e(TAG, "PDF", e);
121     }
122 }
123
124 /**

```

174 / **

第 10 页 / 共 112 页

[illegible]

/ ** *

第 14 页 / 共 112 页

```

425
426         // ■■■■■■■■■■
427         int start = Math.max(0, titleMatcher.start() - 500);
428         int end = Math.min(pdfContent.length(), titleMatcher.end() +
429         String context = pdfContent.substring(start, end);
430
431         int pageIndex = findPageInContext(context);
432
433         if (pageIndex >= 0) {
434             TocOutlineItem item = new TocOutlineItem(title, pageInde
435             outlineItems.add(item);
436             count++;
437         }
438     } catch (Exception e) {
439         Log.w(TAG, "■■■■■■■■", e);
440     }
441 }
442
443 } catch (Exception e) {
444     Log.e(TAG, "■■■■■■■■■■", e);
445 }
446 }
447
448 /**
449  * ■■■■■■■■■■
450  */
451 private static int findPageInContext(String context) {
452     // ■■ /Dest [■■ ■ /Page ■■
453     Pattern destPattern = Pattern.compile("/Dest\\s*\\[\\s*(\\d+)");
454     Matcher destMatcher = destPattern.matcher(context);
455     if (destMatcher.find()) {
456         try {
457             int page = Integer.parseInt(destMatcher.group(1));
458             return Math.max(0, page - 1);
459         } catch (NumberFormatException ignored) {
460         }
461     }
462
463     // ■■ /Page (■■)
464     Pattern pagePattern = Pattern.compile("/Page\\s*\\((\\d+)");
465     Matcher pageMatcher = pagePattern.matcher(context);
466     if (pageMatcher.find()) {
467         try {
468             int page = Integer.parseInt(pageMatcher.group(1));
469             return Math.max(0, page - 1);
470         } catch (NumberFormatException ignored) {
471         }
472     }
473
474     return -1;

```

```

475     }
476 }
477
478 1 package person.notfresh.readingshare.util;
479 2
480 3 import java.io.IOException;
481 4 import java.net.HttpURLConnection;
482 5 import java.net.URL;
483 6
484 7 public class BilibiliUrlConverter {
485 8
486 9     public static String getRedirectedUrl(String shortUrl) throws IOException {
487 10         HttpURLConnection connection = (HttpURLConnection) new URL(shortUrl).openConnection();
488 11         connection.setInstanceFollowRedirects(false);
489 12         connection.connect();
490 13         String redirectedUrl = connection.getHeaderField("Location");
491 14         connection.disconnect();
492 15         return redirectedUrl;
493 16     }
494 17
495 18     public static String convertToBilibiliScheme(String url) {
496 19         if (url == null) {
497 20             return null;
498 21         }
499 22         if (url.startsWith("https://www.bilibili.com/video/")) {
500 23             return url.replace("https://www.bilibili.com/video/", "bilibili://video/");
501 24         }
502 25         return url;
503 26     }
504 27 }
505
506 1 package person.notfresh.readingshare.util;
507 2
508 3 import android.util.Log;
509 4 import android.widget.EditText;
510 5
511 6 import java.io.BufferedReader;
512 7 import java.io.IOException;
513 8 import java.io.InputStream;
514 9
515 10 import org.json.JSONException;
516 11 import org.json.JSONObject;
517 12
518 13 import android.graphics.Bitmap;
519 14 import android.graphics.BitmapFactory;
520 15 import org.jsoup.Jsoup;
521 16 import org.jsoup.nodes.Document;
522 17 import org.jsoup.nodes.Element;
523 18 import org.jsoup.select.Elements;
524 19 import javax.net.ssl.SSLContext;
525 20 import javax.net.ssl.TrustManager;

```

[illegible]

[illegible]

```

121         stringBuilder.append(text).append("\n");
122     }
123 }
124 }
125
126 String webContent = stringBuilder.toString().trim();
127
128 // ■■■■■■■■■■
129 if (webContent.isEmpty()) {
130     webContent = doc.title();
131 }
132
133 return webContent;
134 }
135
136 public static String getUrlSummary(String url, int timeoutSeconds) throws IO
137     // ■■■API■■■URL
138     String apiUrl = Config.API.DUXIANG_API_URL;
139
140     // ■■■■
141     URL apiUrlObj = new URL(apiUrl);
142     HttpURLConnection conn = (HttpURLConnection) apiUrlObj.openConnection();
143
144     // ■■■■■■■■■■
145     if (conn instanceof javax.net.ssl.HttpsURLConnection) {
146         javax.net.ssl.HttpsURLConnection httpsConn = (javax.net.ssl.HttpsURLConnection)
147             javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.Trust
148                 new javax.net.ssl.X509TrustManager() {
149                     public java.security.cert.X509Certificate[] getAcceptedIssuers() {
150                         return null;
151                     }
152                     public void checkClientTrusted(java.security.cert.X509Certif
153                     }
154                     public void checkServerTrusted(java.security.cert.X509Certif
155                     }
156                 }
157     };
158
159     javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("
160     sc.init(null, trustAllCerts, new java.security.SecureRandom());
161     httpsConn.setSSLSocketFactory(sc.getSocketFactory());
162     httpsConn.setHostnameVerifier((hostname, session) -> true);
163 }
164 conn.setRequestMethod("POST");
165 conn.setConnectTimeout(Config.Network.CONNECT_TIMEOUT);
166 conn.setReadTimeout(timeoutSeconds * 1000);
167 conn.setDoOutput(true);
168 conn.setRequestProperty("Content-Type", Config.Headers.CONTENT_TYPE_JSON
169
170 // ■■■■■JSON

```

```

171         JSONObject requestJson = new JSONObject();
172         requestJson.put("url", url);
173         requestJson.put("key", Config.API.DUXIANG_API_KEY);
174
175         // ■■■■■
176         try (java.io.OutputStream os = conn.getOutputStream()) {
177             byte[] input = requestJson.toString().getBytes("utf-8");
178             os.write(input, 0, input.length);
179         }
180
181         // ■■■■
182         StringBuilder response = new StringBuilder();
183         try (BufferedReader reader = new BufferedReader(
184             new InputStreamReader(conn.getInputStream(), StandardCharsets.UTF_8)
185             String line;
186             while ((line = reader.readLine()) != null) {
187                 response.append(line);
188             }
189         )
190         {
191             // ■■JSON■■
192             JSONObject jsonResponse = new JSONObject(response.toString());
193             if (jsonResponse.getBoolean("success")) {
194                 JSONObject data = jsonResponse.getJSONObject("data");
195                 return data.getString("summary");
196             }
197
198             throw new IOException("Failed to get summary from API");
199         }
200
201
202     public static String fetchTitleCommon(String url) throws IOException {
203         String title = "";
204         URL urlObj = new URL(url);
205         URLConnection conn = urlObj.openConnection();
206         conn.setConnectTimeout(3000);
207         conn.setReadTimeout(3000);
208         conn.setRequestProperty("User-Agent",
209             "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (K
210
211         BufferedReader reader = new BufferedReader(
212             new InputStreamReader(conn.getInputStream(), StandardCharsets.UT
213         );
214
215         StringBuilder html = new StringBuilder();
216         String line;
217         int linesRead = 0;
218         while ((line = reader.readLine()) != null && linesRead < 100) {
219             html.append(line);
220             linesRead++;

```

```

221     }
222     reader.close();
223
224     Pattern pattern = Pattern.compile("<title>(.*?)</title>", Pattern.CASE_I
225     Matcher matcher = pattern.matcher(html.toString());
226     if (matcher.find()) {
227         title = matcher.group(1).trim();
228         // ■■■■■■20■■■■■■■20■■■■■■■
229         if (title.length() > 20) {
230             title = title.substring(0, 20) + "...";
231         }
232     }
233     return title;
234 }
235
236 /**
237  * ■■■■■■ favicon URL
238  * ■■■■■■■■■■ HTML ■ head ■■■■■ </head> ■■■
239  * @param url ■■ URL
240  * @return favicon URL■■■■■■■■■■ null
241  */
242 public static String getFaviconUrl(String url) throws IOException {
243     try {
244         URL urlObj = new URL(url);
245         HttpURLConnection conn = (HttpURLConnection) urlObj.openConnection()
246
247         // ■■■■■■■■■■ fetchTitleCommon■
248         conn.setConnectTimeout(3000);
249         conn.setReadTimeout(5000);
250         conn.setRequestProperty("User-Agent", Config.UserAgent.CHROME_USER_A
251         conn.setRequestProperty("Accept", "text/html,application/xhtml+xml,a
252
253         // ■■■■ HTML■■■■■ </head> ■■
254         StringBuilder html = new StringBuilder();
255         try (BufferedReader reader = new BufferedReader(
256             new InputStreamReader(conn.getInputStream(), StandardCharset
257             String line;
258             int linesRead = 0;
259             final int MAX_LINES = 500; // ■■■■ 500 ■
260             final int MAX_SIZE = 50 * 1024; // ■■ 50KB
261
262             while ((line = reader.readLine()) != null && linesRead < MAX_LIN
263                 html.append(line).append("\n");
264                 linesRead++;
265
266             // ■■■■■■ </head> ■■■■■■■■■■
267             String htmlStr = html.toString().toLowerCase();
268             if (htmlStr.contains("</head>")) {
269                 Log.d("CrawlUtil", "Found </head> tag, stopping at line
270                 break;

```

```

271         }
272
273         // ■■■■■■■■
274         if (html.length() > MAX_SIZE) {
275             Log.d("CrawlUtil", "Reached max size limit, stopping");
276             break;
277         }
278     }
279 }
280
281 // ■■ Jsoup ■■ HTML ■■■■■■ head ■■■
282 Document doc = Jsoup.parse(html.toString());
283
284 // ■■■■■■ favicon ■■
285 String faviconUrl = null;
286
287 // 1. ■■ <link rel="icon">
288 Elements iconLinks = doc.select("link[rel=icon]");
289 if (!iconLinks.isEmpty()) {
290     faviconUrl = iconLinks.first().attr("href");
291     Log.d("CrawlUtil", "Found favicon via rel=icon: " + faviconUrl);
292 }
293
294 // 2. ■■■■■■■■ <link rel="shortcut icon">
295 if (faviconUrl == null || faviconUrl.isEmpty()) {
296     Elements shortcutIconLinks = doc.select("link[rel=shortcut icon]");
297     if (!shortcutIconLinks.isEmpty()) {
298         faviconUrl = shortcutIconLinks.first().attr("href");
299         Log.d("CrawlUtil", "Found favicon via rel=shortcut icon: " +
300             faviconUrl);
301     }
302 }
303
304 // 3. ■■■■■■■■ apple-touch-icon■■■■■■■ favicon■
305 if (faviconUrl == null || faviconUrl.isEmpty()) {
306     Elements appleIconLinks = doc.select("link[rel=apple-touch-icon]");
307     if (!appleIconLinks.isEmpty()) {
308         faviconUrl = appleIconLinks.first().attr("href");
309         Log.d("CrawlUtil", "Found favicon via rel=apple-touch-icon: " +
310             faviconUrl);
311     }
312 }
313
314 // ■■■ HTML ■■■■ favicon URL■■■■■■■ URL
315 if (faviconUrl != null && !faviconUrl.isEmpty()) {
316     faviconUrl = resolveUrl(url, faviconUrl);
317     Log.d("CrawlUtil", "Resolved favicon URL: " + faviconUrl);
318     return faviconUrl;
319 }
320
321 // ■■ HTML ■■■■■■■■■■ /favicon.ico
322 String baseUrl = urlObj.getProtocol() + "://" + urlObj.getHost();

```

370

420

第 25 页 / 共 112 页

71 * ██████████CSV██████████

```
121     String fileName = "links_" + getCurrentTime() + "_readshare.json";
```

第 28 页 / 共 112 页

```

172      * @throws JSONException JSON 异常
173      */
174      public static Uri exportToPublicDirectory(Context context, List<LinkItem> li
175          boolean isJson, String fileName) th
176          String mimeType = isJson ? "text/plain" : "text/csv";
177
178      if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
179          // Android 10+ 使用 MediaStore API
180          ContentValues values = new ContentValues();
181          values.put(MediaStore.MediaColumns.DISPLAY_NAME, fileName);
182          values.put(MediaStore.MediaColumns.MIME_TYPE, mimeType);
183          values.put(MediaStore.MediaColumns.RELATIVE_PATH, Environment.DIRECT
184
185          ContentResolver resolver = context.getContentResolver();
186          Uri fileUri = resolver.insert(MediaStore.Files.getContentUri("extern
187
188          if (fileUri != null) {
189              OutputStream outputStream = resolver.openOutputStream(fileUri);
190              if (outputStream != null) {
191                  writeDataToStream(outputStream, links, isJson);
192                  outputStream.close();
193                  return fileUri;
194              }
195          }
196          throw new IOException("写入失败");
197      } else {
198          // Android 9 及以下使用外部存储
199          File documentsFolder = Environment.getExternalStoragePublicDirectory
200              Environment.DIRECTORY_DOCUMENTS);
201          if (!documentsFolder.exists()) {
202              documentsFolder.mkdirs();
203          }
204
205          File outputFile = new File(documentsFolder, fileName);
206          FileOutputStream fos = new FileOutputStream(outputFile);
207          writeDataToStream(fos, links, isJson);
208          fos.close();
209
210          return Uri.fromFile(outputFile);
211      }
212  }
213
214  /**
215   * 将数据写入输出流
216   * @param outputStream 输出流
217   * @param links 数据列表
218   * @param isJson 是否 JSON 格式
219   * @throws IOException 写入失败
220   * @throws JSONException JSON 异常
221   */

```

```

222     private static void writeDataToStream(OutputStream outputStream, List<LinkIt
223                                         boolean isJson) throws IOException, JSO
224     {
225         OutputStreamWriter writer = new OutputStreamWriter(
226             outputStream, StandardCharsets.UTF_8);
227
228         if (isJson) {
229             // ■■ JSON ■■
230             JSONArray jsonArray = new JSONArray();
231             for (LinkItem link : links) {
232                 try {
233                     JSONObject jsonObject = new JSONObject();
234                     jsonObject.put("title", link.getTitle());
235                     jsonObject.put("url", link.getUrl());
236                     jsonObject.put("tags", new JSONArray(link.getTags()));
237                     jsonArray.put(jsonObject);
238                 } catch (JSONException e) {
239                     Log.e("ExportUtil", "Error creating JSON object", e);
240                 }
241             }
242             // ■ JSONObject ■■■■■■■■■■\ / -> /■■■ JSON ■■■
243             String jsonString = jsonArray.toString(4).replace("\\\\", "/");
244             writer.write(jsonString);
245         } else {
246             // ■■ CSV ■■
247             StringBuilder csv = new StringBuilder();
248             csv.append("■■,■■,■■,■■,■■■■,■■\n");
249
250             SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", L
251
252             for (LinkItem link : links) {
253                 String title = escapeCSV(link.getTitle());
254                 String url = escapeCSV(link.getUrl());
255                 String date = sdf.format(new Date(link.getTimestamp()));
256                 String tags = escapeCSV(TextUtils.join(",", link.getTags()));
257                 String clickCount = String.valueOf(link.getClickCount());
258                 String summary = escapeCSV(link.getSummary());
259
260                 csv.append(String.format("%s,%s,%s,%s,%s,%s\n",
261                                         title, url, date, tags, clickCount, summary));
262             }
263             writer.write(csv.toString());
264         }
265
266         writer.flush();
267         writer.close();
268     }
269
270     private static String escapeCSV(String value) {
271         if (value == null) return "";
272         value = value.replace("\"", "\\\""); // ■■■■■■

```

```

272         if (value.contains(",") || value.contains("\") || value.contains("\n"))
273             value = "\"" + value + "\""; // ████████████████████
274     }
275     return value;
276 }
277 }
1  package person.notfresh.readingshare.util;
2
3  import android.content.ContentResolver;
4  import android.content.Context;
5  import android.net.Uri;
6  import android.util.Log;
7  import org.json.JSONArray;
8  import org.json.JSONObject;
9  import org.json.JSONException;
10 import person.notfresh.readingshare.model.LinkItem;
11
12 import java.io.BufferedReader;
13 import java.io.InputStream;
14 import java.io.InputStreamReader;
15 import java.text.ParseException;
16 import java.text.SimpleDateFormat;
17 import java.util.ArrayList;
18 import java.util.Date;
19 import java.util.List;
20
21 /**
22  * ██████████ CSV █ JSON ██████████
23  */
24 public class ImportUtil {
25
26     /**
27      * ████████
28      */
29     public static class ImportResult {
30         public final String format; // "CSV" █ "JSON"
31         public final int successCount;
32         public final int errorCount;
33         public final List<LinkItem> items;
34         public final List<String> errors;
35
36         public ImportResult(String format, int successCount, int errorCount,
37                             List<LinkItem> items, List<String> errors) {
38             this.format = format;
39             this.successCount = successCount;
40             this.errorCount = errorCount;
41             this.items = items;
42             this.errors = errors;
43         }
44

```

```
94     public static ImportResult importFromUri(Context context, Uri uri) {
```

[illegible]

194

```

195         int clickCount = 0;
196         if (columns.length >= 5) {
197             try {
198                 clickCount = Integer.parseInt(columns[4].trim())
199             } catch (NumberFormatException e) {
200                 Log.w("ImportUtil", "■■■■■■■■: " + columns[4], e)
201                 clickCount = 0;
202             }
203         }
204
205         String summary = "";
206         if (columns.length >= 6) {
207             summary = columns[5].trim().replaceFirst("^\\|\\$",
208         }
209
210         LinkItem newLink = new LinkItem(title, url, "imported",
211         newLink.setTimestamp(timestamp);
212         newLink.setTags(tags);
213         newLink.setClickCount(clickCount);
214         newLink.setSummary(summary);
215
216         items.add(newLink);
217         Log.d("ImportUtil", "■■ CSV ■: " + title + ", ■■■■=" + t
218     }
219 } catch (Exception e) {
220     String errorMsg = "■■■■■■■■: " + line + ", ■■: " + e.getMessag
221     Log.e("ImportUtil", errorMsg, e);
222     errors.add(errorMsg);
223 }
224 }
225 reader.close();
226
227     return new ImportResult("CSV", items.size(), errors.size(), items, e
228 } catch (Exception e) {
229     String errorMsg = "■■ CSV ■■: " + e.getMessage();
230     Log.e("ImportUtil", errorMsg, e);
231     errors.add(errorMsg);
232     return new ImportResult("CSV", items.size(), errors.size(), items, e
233 }
234 }
235
236 /**
237  * ■ JSON ■■■■
238  * @param context Context
239  * @param uri JSON ■■ URI
240  * @return ImportResult ■■■■
241  */
242 public static ImportResult importFromJson(Context context, Uri uri) {
243     List<LinkItem> items = new ArrayList<>();
244     List<String> errors = new ArrayList<>();

```

```
245
246     try {
247         InputStream inputStream = context.getContentResolver().openInputStre
248         if (inputStream == null) {
249             errors.add("■■■■■■■■");
250             return new ImportResult("JSON", 0, 1, items, errors);
251         }
252
253         BufferedReader reader = new BufferedReader(new InputStreamReader(inp
254         StringBuilder jsonString = new StringBuilder();
255         String line;
256
257         // ■■■■ JSON ■■
258         while ((line = reader.readLine()) != null) {
259             jsonString.append(line);
260         }
261         reader.close();
262
263         // ■■ JSON
264         JSONArray jsonArray = new JSONArray(jsonString.toString());
265
266         for (int i = 0; i < jsonArray.length(); i++) {
267             try {
268                 JSONObject jsonObject = jsonArray.getJSONObject(i);
269
270                 // ■■■■■■
271                 String title = jsonObject.optString("title", "");
272                 String url = jsonObject.optString("url", "");
273
274                 if (title.isEmpty() || url.isEmpty()) {
275                     String errorMsg = "■■■■■■■■■ title ■ url■: " + jsonObject
276                     Log.w("ImportUtil", errorMsg);
277                     errors.add(errorMsg);
278                     continue;
279                 }
280
281                 // ■■■■■■
282                 long timestamp = jsonObject.optLong("timestamp", System.curr
283                 int clickCount = jsonObject.optInt("clickCount", 0);
284                 String summary = jsonObject.optString("summary", "");
285
286                 // ■■■■
287                 List<String> tags = new ArrayList<>();
288                 if (jsonObject.has("tags")) {
289                     JSONArray tagsArray = jsonObject.optJSONArray("tags");
290                     if (tagsArray != null) {
291                         for (int j = 0; j < tagsArray.length(); j++) {
292                             String tag = tagsArray.optString(j, "");
293                             if (!tag.isEmpty()) {
294                                 tags.add(tag);
295                             }
296                         }
297                     }
298                 }
299             } catch (JSONException e) {
300                 errors.add("■■■■■■■■");
301             }
302         }
303     } catch (IOException e) {
304         errors.add("■■■■■■■■");
305     }
306     return new ImportResult("JSON", 0, 1, items, errors);
307 }
```

```

295         }
296     }
297 }
298 }
299
300 // ■■ LinkItem
301 LinkItem newLink = new LinkItem(title, url, "imported", "",
302 newLink.setTimestamp(timestamp);
303 newLink.setTags(tags);
304 newLink.setClickCount(clickCount);
305 newLink.setSummary(summary);
306
307 items.add(newLink);
308 Log.d("ImportUtil", "■■ JSON ■■: " + title + ", ■■■■=" + tag
309 } catch (JSONException e) {
310 String errorMsg = "■■ JSON ■■■■: " + e.getMessage();
311 Log.e("ImportUtil", errorMsg, e);
312 errors.add(errorMsg);
313 }
314 }
315
316 return new ImportResult("JSON", items.size(), errors.size(), items,
317 } catch (JSONException e) {
318 String errorMsg = "JSON ■■■■: " + e.getMessage();
319 Log.e("ImportUtil", errorMsg, e);
320 errors.add(errorMsg);
321 return new ImportResult("JSON", items.size(), errors.size(), items,
322 } catch (Exception e) {
323 String errorMsg = "■■ JSON ■■: " + e.getMessage();
324 Log.e("ImportUtil", errorMsg, e);
325 errors.add(errorMsg);
326 return new ImportResult("JSON", items.size(), errors.size(), items,
327 }
328 }
329 }
330
331 1 package person.notfresh.readingshare.util;
332 2
333 3 import android.content.Context;
334 4 import android.content.Intent;
335 5 import android.graphics.Bitmap;
336 6 import android.graphics.BitmapFactory;
337 7 import android.net.Uri;
338 8 import android.provider.MediaStore;
339 9 import android.util.Log;
340 10
341 11 import java.io.InputStream;
342 12
343 13 /**
344 14  * ■■■■■■■■

```

```

15  * ████████████████████████████████████████
16  */
17  public class ImageUtil {
18
19      /**
20       * ██████████ Intent██████
21       * @return ██████ Intent
22       */
23      public static Intent createGalleryPickerIntent() {
24          return new Intent(Intent.ACTION_PICK,
25              MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
26      }
27
28      /**
29       * ██████████ Intent██████
30       * @return ████████████████ Intent
31       */
32      public static Intent createGalleryPickerIntentMultiple() {
33          Intent intent = new Intent(Intent.ACTION_GET_CONTENT);
34          intent.setType("image/*");
35          intent.putExtra(Intent.EXTRA_ALLOW_MULTIPLE, true);
36          return intent;
37      }
38
39      /**
40       * █ URI █████ Bitmap
41       * ███ content:// █ file:// ███
42       * @param context ████
43       * @param uri ███ URI
44       * @return Bitmap ████████████████████ null
45       */
46      public static Bitmap uriToBitmap(Context context, Uri uri) {
47          if (context == null || uri == null) {
48              Log.w("ImageUtil", "Context or URI is null");
49              return null;
50          }
51
52          try {
53              InputStream inputStream = context.getContentResolver().openInputStre
54              if (inputStream == null) {
55                  Log.w("ImageUtil", "Failed to open input stream for URI: " + uri
56                  return null;
57              }
58
59              Bitmap bitmap = BitmapFactory.decodeStream(inputStream);
60              inputStream.close();
61
62              if (bitmap != null) {
63                  Log.d("ImageUtil", "Successfully converted URI to Bitmap, size:
64                      bitmap.getWidth() + "x" + bitmap.getHeight());

```

```

65         } else {
66             Log.w("ImageUtil", "Failed to decode bitmap from URI: " + uri);
67         }
68
69         return bitmap;
70     } catch (Exception e) {
71         Log.e("ImageUtil", "Error converting URI to Bitmap: " + e.getMessage);
72         return null;
73     }
74 }
75
76 /**
77  * ■■■■■■■■■■
78  * @param image ■■■■
79  * @param maxSize ■■■■■■■■■■
80  * @return ■■■■■■
81  */
82 public static Bitmap resizeBitmap(Bitmap image, int maxSize) {
83     if (image == null) {
84         Log.w("ImageUtil", "Image is null");
85         return null;
86     }
87
88     int width = image.getWidth();
89     int height = image.getHeight();
90
91     // ■■■■■■■■■■■■■■■■■■■■
92     if (width <= maxSize && height <= maxSize) {
93         return image;
94     }
95
96     float bitmapRatio = (float) width / (float) height;
97     int newWidth, newHeight;
98
99     if (bitmapRatio > 1) {
100         // ■■■■■■
101         newWidth = maxSize;
102         newHeight = (int) (newWidth / bitmapRatio);
103     } else {
104         // ■■■■■■■■
105         newHeight = maxSize;
106         newWidth = (int) (newHeight * bitmapRatio);
107     }
108
109     Log.d("ImageUtil", "Resizing bitmap from " + width + "x" + height +
110         " to " + newWidth + "x" + newHeight);
111
112     return Bitmap.createScaledBitmap(image, newWidth, newHeight, true);
113 }
114

```

```

115     /**
116     * ████████████████████████████████████████
117     * ████████████████████████████████████████
118     * @param image ██████
119     * @param size ████████████████████
120     * @return ████████████
121     */
122     public static Bitmap resizeToSquare(Bitmap image, int size) {
123         if (image == null) {
124             Log.w("ImageUtil", "Image is null");
125             return null;
126         }
127
128         int width = image.getWidth();
129         int height = image.getHeight();
130
131         // ████████████████████████████████████████
132         if (width == size && height == size) {
133             return image;
134         }
135
136         // ████████████████████████████████████████
137         float scale = (float) size / Math.min(width, height);
138         int scaledWidth = (int) (width * scale);
139         int scaledHeight = (int) (height * scale);
140
141         // ████████
142         Bitmap scaledBitmap = Bitmap.createScaledBitmap(image, scaledWidth, scal
143
144         // ████████████████████████████████████████
145         if (scaledWidth != scaledHeight) {
146             int x = (scaledWidth - size) / 2;
147             int y = (scaledHeight - size) / 2;
148
149             // ████████████
150             x = Math.max(0, x);
151             y = Math.max(0, y);
152
153             Bitmap squareBitmap = Bitmap.createBitmap(scaledBitmap, x, y, size,
154
155             // ████████████████████████████████████████
156             if (scaledBitmap != image) {
157                 scaledBitmap.recycle();
158             }
159
160             Log.d("ImageUtil", "Resized to square: " + size + "x" + size);
161             return squareBitmap;
162         }
163
164         Log.d("ImageUtil", "Resized to square: " + size + "x" + size);

```

```

165         return scaledBitmap;
166     }
167
168     /**
169     * ████████████████████████████████████████
170     * ████████████████████████████████████████████████████████████████████████████████
171     * @param image ██████
172     * @param size ████████████████████
173     * @return ████████████
174     */
175     public static Bitmap resizeToSquareForShortcut(Bitmap image) {
176         return resizeToSquare(image, 256); // ████████████████
177     }
178 }
179
180 package person.notfresh.readingshare.util;
181
182 import android.content.Context;
183 import android.widget.Toast;
184
185 import androidx.fragment.app.FragmentManager;
186
187 import java.util.ArrayList;
188 import java.util.List;
189
190 import person.notfresh.readingshare.core.model.SubjectItem;
191 import person.notfresh.readingshare.db.SubjectDao;
192 import person.notfresh.readingshare.model.LinkItem;
193 import person.notfresh.readingshare.ui.subject.SelectSubjectDialog;
194
195 /**
196 * ████████
197 * ████████████████████████████████████████UI██████████
198 */
199 public class SubjectUtil {
200
201     /**
202     * ████████████████████████████████████████████████████████████████████████████████
203     * ████████████████████
204     *
205     * @param context Context███
206     * @param fragmentManager FragmentManager██████████
207     * @param items ████████████
208     */
209     public static void addLinksToSubject(Context context, FragmentManager fragme
210         if (items == null || items.isEmpty()) {
211             Toast.makeText(context, "██████████████████", Toast.LENGTH_SHORT).show();
212             return;
213         }
214 }
215

```

```
/**
 * ████████████████████████████████████
 *
 * @param context Context███
 * @param subjectId ███ID
 * @param linkId ███ID
 */
public static void addLinkToSubjectById(Context context, long subjectId, lon
    SubjectDao subjectDao = new SubjectDao(context);
    subjectDao.open();
    try {
        List<SubjectItem> existingItems = subjectDao.getSubjectItemsBySubjec
```

[illegible]

第 44 页 / 共 112 页

138

```

139         // ■■■■■■■■
140         adapter.archiveOneItem(currentSwipedItem
141         Log.d("SwipeActionsHelperX", "■■■■■■■■■■",
142
143         // ■■■■■■■■
144         adapter.getListener().onDeleteLink(curre
145         Log.d("SwipeActionsHelperX", "■■■■■■■■■■■
146
147         Toast.makeText(rv.getContext(),
148             "■■■■: " + currentSwipedItem.get
149             Toast.LENGTH_SHORT).show();
150         Log.d("SwipeActionsHelperX", "■■■■: " +
151     } else {
152         Toast.makeText(rv.getContext(),
153             "■■■■■■■■■■■■■■■■■■■■",
154             Toast.LENGTH_SHORT).show();
155     }
156
157     closeActiveItem();
158     return true;
159 }
160 } catch (Exception ex) {
161     Log.e(TAG, "Error processing touch: " + ex.getMe
162 }
163 }
164 isTracking = false;
165 break;
166 }
167 return false;
168 }
169
170 @Override
171 public void onTouchEvent(@NonNull RecyclerView rv, @NonNull MotionEvent e) {
172     Log.d(TAG, "onTouchEvent: " + e.getAction());
173 }
174
175 @Override
176 public void onRequestDisallowInterceptTouchEvent(boolean disallowInt
177 });
178 }
179
180 // ■■■■■■■■
181 private void closeActiveItem() {
182     Log.d(TAG, "closeActiveItem: " + activePosition);
183     if (activePosition != -1) {
184         isClosingItem = true;
185         int positionToClose = activePosition;
186         activePosition = -1;
187
188         // ■■■■■■■■

```

```

189         currentSwipedItem = null;
190
191         // ■■■■■■■■■■
192         adapter.notifyItemChanged(positionToClose);
193
194         // ■■■■■■
195         handler.removeCallbacksAndMessages(null);
196
197         // ■■■■■■■■■■■■■■■■■■■■
198         handler.postDelayed(() -> {
199             isClosingItem = false;
200             Log.d(TAG, "Closing animation completed");
201         }, 300);
202     }
203 }
204
205 private ItemTouchHelper.SimpleCallback createSwipeCallback() {
206     return new ItemTouchHelper.SimpleCallback(0, ItemTouchHelper.LEFT) {
207
208         @Override
209         public boolean onMove(@NonNull RecyclerView recyclerView, @NonNull R
210             return false; // ■■■■■■
211     }
212
213     // ■■■■■■
214     @Override
215     public float getSwipeThreshold(@NonNull RecyclerView.ViewHolder view
216         return 0.15f; // ■■■■■■■■■■■■
217     }
218
219     // ■■■■■■■■■■
220     @Override
221     public float getSwipeEscapeVelocity(float defaultValue) {
222         return defaultValue * 0.5f; // ■■■■■■■■■■■■
223     }
224
225     @Override
226     public int getSwipeDirs(@NonNull RecyclerView recyclerView, @NonNull
227         // ■■■■■■■■■■■■■■■■■■■■
228         if (!(viewHolder instanceof LinksAdapter.LinkViewHolder) || adap
229             return 0;
230     }
231
232     // ■■■■■■■■■■
233     return ItemTouchHelper.LEFT;
234 }
235
236 @Override
237 public void onSwiped(@NonNull RecyclerView.ViewHolder viewHolder, in
238     int position = viewHolder.getAdapterPosition());

```

```

239         Log.d(TAG+"X", "onSwiped: position=" + position);
240
241         // ■■■■■■
242         activePosition = position;
243
244         // ■■■■■■LinkItem■■■
245         Object item = adapter.getItemAtPosition(position);
246         if (item instanceof LinkItem) {
247             setCurrentSwipedItem((LinkItem) item);
248             Log.d(TAG+"X", "swap item id is " + ((LinkItem) item).getId(
249         } else {
250             setCurrentSwipedItem(null);
251             Log.w(TAG, "■■■■■LinkItem■■");
252         }
253
254         // ■■■■■■■■■■
255         handler.removeCallbacksAndMessages(null);
256         handler.postDelayed(() -> {
257             Log.d(TAG, "Timer expired for position: " + position);
258             if (activePosition == position) {
259                 closeActiveItem();
260             }
261         }, BUTTONS_DISPLAY_DURATION);
262     }
263
264     @Override
265     public void onChildDraw(@NonNull Canvas c, @NonNull RecyclerView rec
266         @NonNull RecyclerView.ViewHolder viewHolder, float
267         int actionState, boolean isCurrentlyActive) {
268
269         int position = viewHolder.getAdapterPosition();
270         View itemView = viewHolder.itemView;
271
272         // ■■■■■■
273         if (position == activePosition && !isClosingItem) {
274             // ■■■■■■■■■■■■■■■■■■■■■■
275             drawButtons(c, itemView, -buttonWidth);
276
277             // ■■■■■■
278             super.onChildDraw(c, recyclerView, viewHolder, -buttonWidth,
279             return;
280         }
281
282         // ■■■■■■■■■■
283         if (dX < -buttonWidth) {
284             dX = -buttonWidth;
285         }
286
287         // ■■■■■■
288         if (dX < 0) {

```

```

289         drawButtons(c, itemView, dx);
290     }
291
292     super.onChildDraw(c, recyclerView, viewHolder, dx, dy, actionSta
293 }
294
295 // ■■■■■■■■■■■■■■■■■■■■
296 private void drawButtons(Canvas c, View itemView, float dx) {
297     // ■■■■■■■■■■■■■■■■■■■■
298     int backgroundColor;
299     String text;
300
301     if ("archive".equals(userEnv)) {
302         backgroundColor = Color.rgb(139, 0, 0); // ■■■■
303         text = "■■■";
304     } else {
305         backgroundColor = Color.rgb(0, 0, 140); // ■■■■
306         text = "■■■";
307     }
308
309     // ■■■■
310     Paint paint = new Paint();
311     paint.setColor(backgroundColor);
312
313     RectF background = new RectF(
314         itemView.getRight() + dx,
315         itemView.getTop(),
316         itemView.getRight(),
317         itemView.getBottom()
318     );
319
320     c.drawRect(background, paint);
321
322     // ■■■■
323     paint.setColor(Color.WHITE);
324     paint.setTextSize(36);
325
326     // ■■■■■■■■
327     float textWidth = paint.measureText(text);
328     float textX = itemView.getRight() - (Math.abs(dx) / 2) - (textWi
329     float textY = itemView.getTop() + (itemView.getHeight() / 2) + 1
330
331     c.drawText(text, textX, textY, paint);
332 }
333
334 // ■■■■■■■■■■ - ■■■false■■■swipe■■■■■■■■■
335 @Override
336 public boolean isItemViewSwipeEnabled() {
337     return activePosition == -1; // ■■■■■■■■■■■■■■■■■■■■
338 }

```

```
339     };
340 }
341 }
1  package person.notfresh.readingshare.ui.subject;
2
3  import android.content.Intent;
4  import android.graphics.Bitmap;
5  import android.net.Uri;
6  import android.os.Bundle;
7  import android.util.Log;
8  import android.view.LayoutInflater;
9  import android.view.Menu;
10 import android.view.MenuInflater;
11 import android.view.MenuItem;
12 import android.view.View;
13 import androidx.appcompat.widget.Toolbar;
14 import android.view.ViewGroup;
15 import android.widget.Toast;
16
17 import androidx.annotation.NonNull;
18 import androidx.annotation.Nullable;
19 import androidx.fragment.app.Fragment;
20 import androidx.recyclerview.widget.ItemTouchHelper;
21 import androidx.recyclerview.widget.LinearLayoutManager;
22 import androidx.recyclerview.widget.RecyclerView;
23
24 import java.util.List;
25
26 import person.notfresh.readingshare.R;
27 import person.notfresh.readingshare.adapter.SubjectAdapter;
28 import person.notfresh.readingshare.core.model.Subject;
29 import person.notfresh.readingshare.core.model.SubjectUtil;
30 import person.notfresh.readingshare.db.SubjectDao;
31 import person.notfresh.readingshare.ui.subject.CreateSubjectDialog;
32 import person.notfresh.readingshare.ui.subject.SubjectDetailActivity;
33 import person.notfresh.readingshare.util.ImageUtil;
34 import person.notfresh.readingshare.util.ShortcutUtil;
35
36 /**
37  * ██████Fragment
38  */
39 public class SubjectFragment extends Fragment implements SubjectAdapter.OnSubjec
40     private static final String TAG = "SubjectFragment";
41     private static final int REQUEST_CODE_PICK_ICON = 1001;
42
43     private RecyclerView recyclerView;
44     private SubjectAdapter adapter;
45     private SubjectDao subjectDao;
46     private Subject pendingIconSubject; // ████████████████
47     private boolean isSortMode = false;
```

```
48     private ItemTouchHelper itemTouchHelper;
49     private MenuItem sortMenuItem;
50
51     @Override
52     public void onCreate(@Nullable Bundle savedInstanceState) {
53         super.onCreate(savedInstanceState);
54         setHasOptionsMenu(true);
55     }
56
57     @Override
58     public void onCreateOptionsMenu(@NonNull Menu menu, @NonNull MenuInflater inflater) {
59         inflater.inflate(R.menu.subject_menu, menu);
60         sortMenuItem = menu.findItem(R.id.action_sort_subjects);
61         super.onCreateOptionsMenu(menu, inflater);
62     }
63
64     @Override
65     public boolean onOptionsItemSelected(@NonNull MenuItem item) {
66         if (item.getItemId() == R.id.action_create_subject) {
67             showCreateSubjectDialog();
68             return true;
69         } else if (item.getItemId() == R.id.action_sort_subjects) {
70             toggleSortMode();
71             return true;
72         }
73         return super.onOptionsItemSelected(item);
74     }
75
76     private void toggleSortMode() {
77         isSortMode = !isSortMode;
78         adapter.setSortMode(isSortMode);
79
80         if (isSortMode) {
81             // ■■■■■■
82             sortMenuItem.setTitle("■■■");
83             Toolbar toolbar = requireActivity().findViewById(R.id.toolbar);
84             if (toolbar != null) {
85                 toolbar.setTitle("■■■...");
86             }
87             itemTouchHelper.attachToRecyclerView(recyclerView);
88         } else {
89             // ■■■■■■
90             sortMenuItem.setTitle("■■■");
91             Toolbar toolbar = requireActivity().findViewById(R.id.toolbar);
92             if (toolbar != null) {
93                 toolbar.setTitle(getString(R.string.menu_subject));
94             }
95             itemTouchHelper.attachToRecyclerView(null);
96         }
97     }
```

```

98
99     private void showCreateSubjectDialog() {
100         CreateSubjectDialog dialog = CreateSubjectDialog.newInstance();
101         dialog.setOnSubjectSavedListener(subject -> {
102             // ■■■■■■■■
103             subjectDao.insertSubject(subject);
104             // ■■■■
105             loadSubjects();
106             Toast.makeText(requireContext(), "■■■■■■■", Toast.LENGTH_SHORT).show(
107         });
108         dialog.show(getParentFragmentManager(), "CreateSubjectDialog");
109     }
110
111     @Override
112     public View onCreateView(@NonNull LayoutInflater inflater,
113                             ViewGroup container, Bundle savedInstanceState) {
114         View root = inflater.inflate(R.layout.fragment_subject, container, false
115
116         subjectDao = new SubjectDao(requireContext());
117         subjectDao.open();
118
119         recyclerView = root.findViewById(R.id.recycler_view);
120         adapter = new SubjectAdapter(requireContext());
121         adapter.setOnSubjectClickListener(this);
122         adapter.setOnSubjectActionListener(new SubjectAdapter.OnSubjectActionLis
123             @Override
124             public void onEditSubject(Subject subject) {
125                 showEditSubjectDialog(subject);
126             }
127
128             @Override
129             public void onDeleteSubject(Subject subject) {
130                 // ■■■■■■■■■■■■■■■■■■■■■■
131                 boolean deleted = subjectDao.deleteSubject(subject.getId());
132                 if (deleted) {
133                     loadSubjects();
134                     Toast.makeText(requireContext(), "■■■■■", Toast.LENGTH_SHORT
135                 } else {
136                     Toast.makeText(requireContext(), "■■■■■", Toast.LENGTH_SHORT)
137                 }
138             }
139
140             @Override
141             public void onAddToDesktop(Subject subject) {
142                 // ■■■■■■■■■■■■■■■■■■■■■■ SubjectAdapter ■■■■■■■■■■■■■■■■■■■■■■
143                 // ■■■■■■■■■■■■■■■■■■■■■■
144                 createSubjectShortcut(subject);
145             }
146
147             @Override

```

```
148         public void onRequestCustomIcon(Subject subject) {
149             Log.d(TAG, "onRequestCustomIcon: " + subject.getTitle());
150
151             // ■■■■■■
152             pendingIconSubject = subject;
153
154             // ■■■■■■■■
155             Intent intent = ImageUtil.createGalleryPickerIntent();
156             startActivityForResult(intent, REQUEST_CODE_PICK_ICON);
157         }
158     });
159     recyclerView.setAdapter(adapter);
160     recyclerView.setLayoutManager(new LinearLayoutManager(requireContext()))
161
162     setupDragAndDrop();
163
164     loadSubjects();
165
166     return root;
167 }
168
169 @Override
170 public void onDestroyView() {
171     super.onDestroyView();
172     if (subjectDao != null) {
173         subjectDao.close();
174     }
175 }
176
177 private void loadSubjects() {
178     List<Subject> subjects = subjectDao.getAllSubjects();
179     adapter.setSubjects(subjects);
180     Log.d(TAG, "■■■■ " + subjects.size() + " ■■■");
181 }
182
183 private void setupDragAndDrop() {
184     ItemTouchHelper.Callback callback = new ItemTouchHelper.SimpleCallback(
185         ItemTouchHelper.UP | ItemTouchHelper.DOWN, 0) {
186         @Override
187         public boolean onMove(@NonNull RecyclerView recyclerView,
188                             @NonNull RecyclerView.ViewHolder viewHolder,
189                             @NonNull RecyclerView.ViewHolder target) {
190             int fromPos = viewHolder.getAdapterPosition();
191             int toPos = target.getAdapterPosition();
192
193             List<Subject> items = adapter.getSubjects();
194             if (fromPos < 0 || fromPos >= items.size() || toPos < 0 || toPos
195                 return false;
196         }
197     }
```

[illegible]

```

248
249  /**
250   * ██████████
251   */
252  private void createSubjectShortcut(Subject subject) {
253      String title = subject.getTitle() != null ? subject.getTitle() : "██";
254      ShortcutUtil.createSubjectShortcut(requireContext(), title, subject.getI
255  }
256
257  @Override
258  public void onActivityResult(int requestCode, int resultCode, Intent data) {
259      super.onActivityResult(requestCode, resultCode, data);
260
261      if (requestCode == REQUEST_CODE_PICK_ICON && resultCode == android.app.A
262          if (data != null && data.getData() != null && pendingIconSubject !=
263              Uri selectedImageUri = data.getData();
264              Log.d(TAG, "██████: " + selectedImageUri);
265
266          try {
267              // 1. █ URI ███ Bitmap
268              Bitmap bitmap = ImageUtil.uriToBitmap(requireContext(), sele
269
270              if (bitmap != null) {
271                  // 2. ████████████████████ 256x256█
272                  Bitmap squareBitmap = ImageUtil.resizeToSquareForShortcu
273
274                  // 3. ████████
275                  if (squareBitmap != bitmap) {
276                      bitmap.recycle();
277                  }
278
279                  // 4. ████████
280                  adapter.createShortcutWithCustomIcon(
281                      requireContext(),
282                      pendingIconSubject,
283                      squareBitmap
284                  );
285
286                  Log.d(TAG, "████████████████████");
287              } else {
288                  Toast.makeText(requireContext(), "███████", Toast.LENGTH_
289                  Log.e(TAG, "███ URI ████");
290              }
291          } catch (Exception e) {
292              Log.e(TAG, "███████: " + e.getMessage(), e);
293              Toast.makeText(requireContext(), "███████: " + e.getMessage(
294          } finally {
295              // ████████
296              pendingIconSubject = null;
297          }

```

```

298         } else {
299             Log.w(TAG, "■■■■■■■■data ■ pendingIconSubject ■ null");
300             pendingIconSubject = null;
301         }
302     }
303 }
304 }
305
306 1 package person.notfresh.readingshare.ui.document;
307 2
308 3 import android.content.Intent;
309 4 import android.content.pm.ActivityInfo;
310 5 import android.graphics.Bitmap;
311 6 import android.graphics.pdf.PdfRenderer;
312 7 import android.net.Uri;
313 8 import android.os.Bundle;
314 9 import android.os.ParcelFileDescriptor;
315 10 import android.util.Log;
316 11 import android.view.GestureDetector;
317 12 import android.view.MotionEvent;
318 13 import android.view.Menu;
319 14 import android.view.MenuItem;
320 15 import android.view.View;
321 16 import android.widget.Button;
322 17 import android.widget.EditText;
323 18 import android.widget.ImageView;
324 19 import android.widget.LinearLayout;
325 20 import android.widget.ScrollView;
326 21 import android.widget.TextView;
327 22 import android.widget.Toast;
328 23
329 24 import androidx.appcompat.app.AppCompatActivity;
330 25 import androidx.appcompat.app.AlertDialog;
331 26 import androidx.appcompat.widget.Toolbar;
332 27 import androidx.drawerlayout.widget.DrawerLayout;
333 28 import androidx.recyclerview.widget.LinearLayoutManager;
334 29 import androidx.recyclerview.widget.RecyclerView;
335 30
336 31 import java.io.File;
337 32 import java.io.FileOutputStream;
338 33 import java.io.InputStream;
339 34 import java.text.SimpleDateFormat;
340 35 import java.util.ArrayList;
341 36 import java.util.Date;
342 37 import java.util.List;
343 38 import java.util.Locale;
344 39
345 40 import person.notfresh.readingshare.R;
346 41 import person.notfresh.readingshare.db.DocumentDao;
347 42 import person.notfresh.readingshare.model.BookmarkItem;

```


第 58 页 / 共 112 页

```

143         Log.w(TAG, "■■■■■URI■■■■■", e);
144     }
145 }
146
147 if (Intent.ACTION_VIEW.equals(action) && data != null) {
148     // ■■■■■PDF■■■
149     isExternalOpen = true;
150     originalUri = data;
151     handleExternalPdf(data);
152 } else if (Intent.ACTION_SEND.equals(action) && data != null) {
153     // ■■■PDF■■■
154     isExternalOpen = true;
155     originalUri = data;
156     handleSharedPdf(data);
157 } else {
158     // ■■■■■■■■■■■document_id■
159     isExternalOpen = false;
160     handleInternalPdf(intent);
161 }
162
163 // ■■■■■■■■
164 btnPrev.setOnClickListener(new View.OnClickListener() {
165     @Override
166     public void onClick(View v) {
167         showPreviousPage();
168     }
169 });
170
171 btnNext.setOnClickListener(new View.OnClickListener() {
172     @Override
173     public void onClick(View v) {
174         showNextPage();
175     }
176 });
177
178 btnToc.setOnClickListener(new View.OnClickListener() {
179     @Override
180     public void onClick(View v) {
181         openTocDrawer();
182     }
183 });
184
185 btnCloseToc.setOnClickListener(new View.OnClickListener() {
186     @Override
187     public void onClick(View v) {
188         closeTocDrawer();
189     }
190 });
191
192 btnViewPages.setOnClickListener(new View.OnClickListener() {

```

```
193         @Override
194         public void onClick(View v) {
195             switchToPageView();
196         }
197     });
198
199     btnViewOutline.setOnClickListener(new View.OnClickListener() {
200         @Override
201         public void onClick(View v) {
202             switchToOutlineView();
203         }
204     });
205
206     btnViewBookmarks.setOnClickListener(new View.OnClickListener() {
207         @Override
208         public void onClick(View v) {
209             switchToBookmarkView();
210         }
211     });
212
213     // ■■■■■/■■■■■
214     setupDoubleTapGesture();
215 }
216
217 private void setupDoubleTapGesture() {
218     ScrollView scrollView = findViewById(R.id.content_scroll_view);
219     if (scrollView == null) return;
220
221     toolbarVisible = toolbar.getVisibility() == View.VISIBLE;
222
223     gestureDetector = new GestureDetector(this, new GestureDetector.SimpleOn
224         @Override
225         public boolean onDoubleTap(MotionEvent e) {
226             toggleToolbar();
227             return true;
228         }
229     });
230
231     scrollView.setOnTouchListener((v, event) -> gestureDetector.onTouchEvent
232 }
233
234 private void toggleToolbar() {
235     if (toolbar == null) return;
236     toolbarVisible = !toolbarVisible;
237     toolbar.setVisibility(toolbarVisible ? View.VISIBLE : View.GONE);
238 }
239
240 @Override
241 public boolean onCreateOptionsMenu(Menu menu) {
242     getMenuInflater().inflate(R.menu.document_viewer_menu, menu);
```

第 61 页 / 共 112 页

```

293     }
294
295     /**
296      * ██████████PDF████document_id█
297      */
298     private void handleInternalPdf(Intent intent) {
299         long documentId = intent.getLongExtra("document_id", -1);
300         if (documentId == -1) {
301             Toast.makeText(this, "██ID██", Toast.LENGTH_SHORT).show();
302             finish();
303             return;
304         }
305
306         document = documentDao.getDocumentById(documentId);
307         if (document == null) {
308             Toast.makeText(this, "██████", Toast.LENGTH_SHORT).show();
309             finish();
310             return;
311         }
312
313         // ████████
314         document.incrementClickCount();
315         documentDao.updateClickCount(document.getId(), document.getClickCount());
316
317         // ████
318         setTitle(document.getTitle());
319
320         // ██PDF
321         pdfFile = new File(document.getFilePath());
322         // ████████████████
323         isSavedToDocuments = true;
324         loadPdf();
325     }
326
327     /**
328      * ██████████PDF████URI█
329      */
330     private void handleExternalPdf(Uri uri) {
331         Log.d(TAG, "████PDF: " + uri.toString());
332
333         try {
334             // ████████
335             String fileName = getFileName(uri);
336             if (fileName == null || fileName.isEmpty()) {
337                 fileName = "external_" + System.currentTimeMillis() + ".pdf";
338             }
339
340             // ████████████████
341             String destPath = copyFileToPrivateDir(uri, fileName);
342             if (destPath == null) {

```

```
/**
 * ██████████PDF████
 */
private void handleSharedPdf(Uri uri) {
    handleExternalPdf(uri);
}
```

第 64 页 / 共 112 页

第 65 页 / 共 112 页

```

493         DocumentItem existing = findExistingDocument(filePath);
494         if (existing != null) {
495             return existing;
496         }
497
498         // ■■■■■■■■■■
499         File file = new File(filePath);
500         DocumentItem item = new DocumentItem(fileName, filePath, DocumentType.PD
501         item.setFileSize(file.length());
502         long id = documentDao.insertDocument(item);
503         item.setId(id);
504         return item;
505     }
506
507     /**
508     * ■■■PDF■■■
509     */
510     private void loadPdf() {
511         if (pdfFile == null || !pdfFile.exists()) {
512             Toast.makeText(this, "■■■■■", Toast.LENGTH_SHORT).show();
513             finish();
514             return;
515         }
516
517         try {
518             // ■■■PDF■■■
519             fileDescriptor = ParcelFileDescriptor.open(pdfFile, ParcelFileDescri
520             pdfRenderer = new PdfRenderer(fileDescriptor);
521             totalPages = pdfRenderer.getPageCount();
522
523             Log.d(TAG, "PDF■■■■■■■ " + totalPages + " ■");
524
525             // ■■■■■■■■■■
526             int savedPage = -1;
527             if (document != null && document.getId() > 0) {
528                 savedPage = documentDao.getReadingProgress(document.getId());
529             }
530             if (savedPage > 0 && savedPage < totalPages) {
531                 currentPageIndex = savedPage;
532             } else {
533                 currentPageIndex = 0;
534             }
535             showPage(currentPageIndex);
536
537             // ■■■■■■
538             initToc();
539
540         } catch (Exception e) {
541             Log.e(TAG, "■■■PDF■■■", e);
542             Toast.makeText(this, "■■■■■PDF■■■: " + e.getMessage(), Toast.LENGTH_SH

```

```

543         finish();
544     }
545 }
546
547 /**
548  * ■■■■■■
549  */
550 private void initToc() {
551     if (totalPages <= 0) {
552         return;
553     }
554
555     // ■■■■■■
556     List<Integer> pageNumbers = new ArrayList<>();
557     for (int i = 1; i <= totalPages; i++) {
558         pageNumbers.add(i);
559     }
560
561     // ■■■■■PDF■■■■■PDF■■■■■
562     extractPdfOutline();
563
564     // ■■■■■■
565     tocAdapter = new TocAdapter(pageNumbers, currentPageIndex);
566     tocAdapter.setOutlineItems(outlineItems);
567     tocAdapter.setOnPageClickListener(new TocAdapter.OnPageClickListener() {
568         @Override
569         public void onPageClick(int pageIndex) {
570             showPage(pageIndex);
571             closeTocDrawer();
572         }
573     });
574
575     tocAdapter.setOnBookmarkDeleteCallback(bookmarkId -> {
576         documentDao.deleteBookmark(bookmarkId);
577         // ■■■■■■
578         List<BookmarkItem> updated = documentDao.getBookmarksByDocument(docu
579         tocAdapter.setBookmarks(updated);
580         Toast.makeText(DocumentViewerActivity.this, "■■■■■", Toast.LENGTH_SH
581         invalidateOptionsMenu());
582     });
583
584     // ■■■■■■
585     if (document != null && document.getId() > 0) {
586         tocAdapter.setBookmarks(documentDao.getBookmarksByDocument(document.
587     }
588
589     recyclerViewToc.setLayoutManager(new LinearLayoutManager(this));
590     recyclerViewToc.setAdapter(tocAdapter);
591
592     // ■■■■■■■■

```

[illegible]

```
643     private void switchToOutlineView() {
644         if (tocAdapter != null) {
645             tocAdapter.setPageView(false);
646             btnViewPages.setEnabled(true);
647             btnViewOutline.setEnabled(false);
648             btnViewBookmarks.setEnabled(true);
649         }
650     }
651
652     private void switchToBookmarkView() {
653         if (tocAdapter != null) {
654             tocAdapter.setViewType(TocAdapter.ViewType.BOOKMARK);
655             btnViewPages.setEnabled(true);
656             btnViewOutline.setEnabled(true);
657             btnViewBookmarks.setEnabled(false);
658         }
659     }
660
661     /**
662     * ████████
663     */
664     private void openTocDrawer() {
665         if (drawerLayout != null) {
666             drawerLayout.openDrawer(findViewById(R.id.drawer_content));
667         }
668     }
669
670     /**
671     * ████████
672     */
673     private void closeTocDrawer() {
674         if (drawerLayout != null) {
675             drawerLayout.closeDrawer(findViewById(R.id.drawer_content));
676         }
677     }
678
679     /**
680     * ████████
681     */
682     private void showPage(int pageIndex) {
683         if (pdfRenderer == null || pageIndex < 0 || pageIndex >= totalPages) {
684             return;
685         }
686
687         try {
688             // ████████
689             if (currentPage != null) {
690                 currentPage.close();
691             }
692         }
```

```

693 // ■■■■■■
694 currentPage = pdfRenderer.openPage(pageIndex);
695 currentPageIndex = pageIndex;
696
697 // ■■■■■■■■■■■■
698 int width = getResources().getDisplayMetrics().widthPixels;
699 int height = (int) (width * 1.414f); // A4■■■■■■■■■■■■
700
701 // ■■■■■■Bitmap
702 Bitmap bitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_4444);
703 bitmap.eraseColor(android.graphics.Color.WHITE);
704 currentPage.render(bitmap, null, null, PdfRenderer.Page.RENDER_MODE_
705
706 // ■■Bitmap
707 pdfImageView.setImageBitmap(bitmap);
708
709 // ■■■■■■
710 updatePageInfo();
711
712 // ■■■■■■
713 btnPrev.setEnabled(pageIndex > 0);
714 btnNext.setEnabled(pageIndex < totalPages - 1);
715
716 // ■■■■■■
717 if (tocAdapter != null) {
718     tocAdapter.setCurrentPage(pageIndex);
719 }
720
721 // ■■■■■■
722 if (document != null && document.getId() > 0) {
723     documentDao.saveReadingProgress(document.getId(), currentPageInd
724 }
725
726 // ■■■■■■■■■■■■
727 invalidateOptionsMenu();
728
729 } catch (Exception e) {
730     Log.e(TAG, "■■■■■■■", e);
731     Toast.makeText(this, "■■■■■■■", Toast.LENGTH_SHORT).show();
732 }
733 }
734
735 /**
736 * ■■■■■■
737 */
738 private void showPreviousPage() {
739     if (currentPageIndex > 0) {
740         showPage(currentPageIndex - 1);
741     }
742 }

```

```

743
744  /**
745   * ██████
746   */
747  private void showNextPage() {
748      if (currentPageIndex < totalPages - 1) {
749          showPage(currentPageIndex + 1);
750      }
751  }
752
753  /**
754   * ██████████
755   */
756  private void updatePageInfo() {
757      if (tvPageInfo != null) {
758          tvPageInfo.setText(String.format(Locale.getDefault(), "%d / %d",
759              currentPageIndex + 1, totalPages));
760      }
761
762      // █████
763      if (document != null) {
764          setTitle(document.getTitle() + " (" + (currentPageIndex + 1) + "/" +
765              document.getPageCount());
766      }
767
768  /**
769   * ██████████
770   */
771  private void saveToDocuments() {
772      if (pdfFile == null || !pdfFile.exists()) {
773          Toast.makeText(this, "██████████████", Toast.LENGTH_SHORT).show();
774          return;
775      }
776
777      try {
778          String filePath = pdfFile.getAbsolutePath();
779          String fileName = pdfFile.getName();
780
781          // ████████████████████████████████
782          if (!filePath.contains(getFilesDir().getAbsolutePath())) {
783              // █████URI██████
784              if (originalUri != null) {
785                  String destPath = copyFileToPrivateDir(originalUri, fileName);
786                  if (destPath != null) {
787                      filePath = destPath;
788                      pdfFile = new File(filePath);
789                  } else {
790                      Toast.makeText(this, "██████████████", Toast.LENGTH_SHORT).
791                          show();
792                      return;
793                  }
794              }
795          }
796      } catch (Exception e) {
797          Toast.makeText(this, "██████████████", Toast.LENGTH_SHORT).show();
798      }
799  }

```

```

793         } else {
794             // ■■■■■■■■■■
795             File documentsDir = new File(getFilesDir(), "documents");
796             File pdfDir = new File(documentsDir, "pdf");
797             if (!pdfDir.exists()) {
798                 pdfDir.mkdirs();
799             }
800
801             String timestamp = new SimpleDateFormat("yyyyMMdd_HHmss", L
802                 .format(new Date()));
803             String destFileName = timestamp + "_" + fileName;
804             File destFile = new File(pdfDir, destFileName);
805
806             // ■■■■
807             java.io.FileInputStream fis = new java.io.FileInputStream(pd
808             FileOutputStream fos = new FileOutputStream(destFile);
809             byte[] buffer = new byte[4096];
810             int bytesRead;
811             while ((bytesRead = fis.read(buffer)) != -1) {
812                 fos.write(buffer, 0, bytesRead);
813             }
814             fis.close();
815             fos.close();
816
817             filePath = destFile.getAbsolutePath();
818             pdfFile = destFile;
819         }
820     }
821
822     // ■■■■■■■■■■
823     java.util.List<DocumentItem> allDocs = documentDao.getAllDocuments()
824     for (DocumentItem doc : allDocs) {
825         if (doc.getFilePath().equals(filePath)) {
826             // ■■■■■■■■■■
827             document = doc;
828             document.incrementClickCount();
829             documentDao.updateClickCount(document.getId(), document.getClickCount() + 1);
830             document.isSavedToDocuments = true;
831             invalidateOptionsMenu();
832             Toast.makeText(this, "■■■■■■■■■■■■■■■■■■■■", Toast.LENGTH_SHORT).show();
833             return;
834         }
835     }
836
837     // ■■■■■■■■■■
838     DocumentItem item = new DocumentItem(fileName, filePath, DocumentType.PDF);
839     item.setFileSize(pdfFile.length());
840     long id = documentDao.insertDocument(item);
841     item.setId(id);
842     document = item;

```

```

843         isSavedToDocuments = true;
844         invalidateOptionsMenu();
845
846         Toast.makeText(this, "■■■■■■■■■", Toast.LENGTH_SHORT).show();
847         Log.d(TAG, "■■■■■■: " + filePath);
848
849     } catch (Exception e) {
850         Log.e(TAG, "■■■■■■■", e);
851         Toast.makeText(this, "■■■■■■: " + e.getMessage(), Toast.LENGTH_SHORT).
852     }
853 }
854
855 /**
856 * ■■■■■■■■/■■■■■■■
857 */
858 private void showBookmarkDialog() {
859     if (document == null || document.getId() <= 0) {
860         Toast.makeText(this, "■■■■■■■■■■■■■■■■■■■■", Toast.LENGTH_SHORT).show();
861         return;
862     }
863
864     // ■■■■■■■■■■■■■■■■■■■■
865     List<BookmarkItem> bookmarks = documentDao.getBookmarksByDocument(docume
866     BookmarkItem existingBookmark = null;
867     for (BookmarkItem bm : bookmarks) {
868         if (bm.getPageIndex() == currentPageIndex) {
869             existingBookmark = bm;
870             break;
871         }
872     }
873
874     boolean isEdit = existingBookmark != null;
875     final BookmarkItem bookmarkToDelete = existingBookmark;
876
877     AlertDialog.Builder builder = new AlertDialog.Builder(this);
878     builder.setTitle(isEdit ? "■■■■■" : "■■■■■");
879
880     View dialogView = getLayoutInflater().inflate(R.layout.dialog_bookmark,
881     final EditText etNote = dialogView.findViewById(R.id.et_bookmark_note);
882     TextView tvHint = dialogView.findViewById(R.id.tv_bookmark_hint);
883     tvHint.setText("■■■■ " + (currentPageIndex + 1) + " ■■■■■■■■■■■■■■■■■■■■■");
884     if (isEdit && existingBookmark.getNote() != null) {
885         etNote.setText(existingBookmark.getNote());
886     }
887     builder.setView(dialogView);
888
889     builder.setPositiveButton("■■■", (dialog, which) -> {
890         String note = etNote.getText().toString().trim();
891         if (note.isEmpty()) {
892             note = null;

```

第74页 / 共112页

```

943         }
944
945         if (fileDescriptor != null) {
946             try {
947                 fileDescriptor.close();
948             } catch (Exception e) {
949                 Log.e(TAG, "■■■■■■■■■■", e);
950             }
951             fileDescriptor = null;
952         }
953
954         if (documentDao != null) {
955             documentDao.close();
956         }
957     }
958 }
1  package person.notfresh.readingshare;
2
3  import static person.notfresh.readingshare.WebViewManager.*;
4
5  import android.content.ClipboardManager;
6  import android.content.ClipData;
7  import android.content.Intent;
8  import android.net.Uri;
9  import android.os.Bundle;
10 import android.os.Handler;
11 import android.os.Looper;
12 import android.util.Log;
13 import android.view.Menu;
14 import android.view.MenuItem;
15 import android.view.MotionEvent;
16 import android.webkit.WebChromeClient;
17 import android.webkit.WebResourceRequest;
18 import android.webkit.WebSettings;
19 import android.webkit.WebView;
20 import android.webkit.WebViewClient;
21 import android.widget.Toast;
22 import androidx.appcompat.app.AlertDialog;
23 import androidx.appcompat.app.AppCompatActivity;
24 import androidx.appcompat.widget.Toolbar;
25 import android.view.View;
26 import android.widget.Button;
27 import android.widget.FrameLayout;
28 import android.content.SharedPreferences;
29 import android.support.v4.media.session.MediaSessionCompat;
30 import android.support.v4.media.session.PlaybackStateCompat;
31 import android.media.AudioAttributes;
32 import android.media.AudioFocusRequest;
33 import android.media.AudioManager;
34 import android.os.PowerManager;

```

```

35 import android.content.Context;
36 import android.os.Build;
37 import android.Manifest;
38 import androidx.core.app.ActivityCompat;
39 import androidx.core.content.ContextCompat;
40 import android.content.pm.PackageManager;
41 import android.webkit.PermissionRequest;
42 import android.view.LayoutInflater;
43 import android.view.ViewGroup;
44
45 import java.io.IOException;
46 import java.net.URISyntaxException;
47 import java.util.ArrayList;
48 import java.util.List;
49 import java.util.concurrent.ExecutorService;
50 import java.util.concurrent.Executors;
51
52 import android.widget.EditText;
53 import android.widget.TextView;
54 import android.widget.LinearLayout;
55 import com.google.android.flexbox.FlexboxLayout;
56
57 import person.notfresh.readingshare.db.LinkDao;
58 import person.notfresh.readingshare.model.LinkItem;
59 import person.notfresh.readingshare.util.CrawlUtil;
60 import person.notfresh.readingshare.util.RecentTagsManager;
61 import person.notfresh.readingshare.util.ShareUtil;
62
63 public class WebViewActivity extends AppCompatActivity {
64     private WebView webView;
65     private Toolbar toolbar;
66     private String currentUrl;
67     private boolean audioPlaying = false;
68     private boolean isAppInForeground = true;
69     private MediaSessionCompat mediaSession;
70     private PowerManager.WakeLock wakeLock;
71     private AudioManager audioManager;
72     private AudioFocusRequest audioFocusRequest;
73     private boolean preserveCache = false;
74     private String pageTitleCache = "";
75     private boolean isExternalOpen = false; // ■■■■■■
76     private long[] contextIds;
77     private int currentIndex;
78     private boolean isNavigating = false;
79     private FrameLayout navigationControls;
80     private Button buttonPrevious;
81     private Button buttonNext;
82     private Handler controlsHandler = new Handler(Looper.getMainLooper());
83     private Runnable hideControlsRunnable;
84     private static final int CONTROLS_AUTO_HIDE_MS = 3000;

```

134

184

第 79 页 / 共 112 页

```
284 @Override
```

```

285     protected void onNewIntent(Intent intent) {
286         super.onNewIntent(intent);
287         setIntent(intent);
288         parseNavigationContext();
289         initControlsIfNeeded();
290         handleNavigationIntent(intent);
291     }
292
293     @Override
294     protected void onResume() {
295         super.onResume();
296         isAppInForeground = true;
297         initControlsIfNeeded();
298         if (webView != null) {
299             webView.onResume();
300             webView.resumeTimers();
301         }
302         // ■■■■■■■■■■■■■■■■■■■■■■
303         stopService(new Intent(this, WebViewBackgroundService.class));
304         if (wakeLock != null && wakeLock.isHeld()) {
305             wakeLock.release();
306         }
307     }
308
309
310     @Override
311     public boolean onOptionsItemSelected(MenuItem item) {
312         if (item.getItemId() == R.id.action_refresh) {
313             if (webView != null) webView.reload();
314             return true;
315         } else if (item.getItemId() == R.id.action_open_browser) {
316             // ■■■■■■
317             String menuItemTitle = item.getTitle().toString();
318             Log.d("WebViewActivityMenu", "■■■■■■■: " + menuItemTitle);
319
320             try {
321                 Intent intent = new Intent(Intent.ACTION_VIEW);
322                 intent.setData(Uri.parse(webView.getUrl()));
323                 startActivity(intent);
324                 return true;
325             } catch (Exception e) {
326                 Toast.makeText(this, "■■■■■■■", Toast.LENGTH_SHORT).show();
327                 return true;
328             }
329         } else if (item.getItemId() == R.id.action_force_back) {
330             // ■■■■■■■■■■■■■■ WebView
331             Log.d("WebViewActivityMenu", "■■■■■■■");
332             finish();
333             return true;
334         } else if (item.getItemId() == R.id.action_archive_back) {

```

```

335 // ■■■■■■■■■■WebView■■■■■■■
336 Log.d("WebViewActivityMenu", "■■■■■■■■");
337 Toast.makeText(this, "■■■■■■", Toast.LENGTH_SHORT).show();
338
339 // ■■■■■■■■
340 if (audioManager != null && audioFocusRequest != null) {
341     AudioManager.abandonAudioFocusRequest(audioFocusRequest);
342 }
343 stopService(new Intent(this, WebViewBackgroundService.class));
344 if (wakeLock != null && wakeLock.isHeld()) {
345     wakeLock.release();
346 }
347 if (mediaSession != null) {
348     mediaSession.setActive(false);
349     mediaSession.release();
350     mediaSession = null;
351 }
352
353 // ■■■■■■■■WebView■■■■■■■
354 if (webView != null && webView.getParent() != null) {
355     ((ViewGroup) webView.getParent()).removeView(webView);
356     // ■■WebView■■
357     getInstance().storeWebView(currentUrl, webView);
358     // ■■webView■null■■■■onDestroy■■■■
359     webView = null;
360 }
361 preserveCache = true; // ■■■■■■
362 finish();
363 return true;
364 } else if (item.getItemId() == R.id.webview_share) {
365     Log.d("WebViewActivityMenu", "■■■■■■");
366     String title = (pageTitleCache != null && !pageTitleCache.isEmpty()) ? pageTitleCache : "";
367     String url = webView != null ? webView.getUrl() : "";
368     Log.d("ShareUtil", "WebViewActivityMenu share title=" + title + ", u" + url);
369     ShareUtil.shareText(this, title, null, url);
370     return true;
371 } else if (item.getItemId() == R.id.action_extract_content) {
372     Log.d("WebViewActivityMenu", "click extract");
373     String url = webView != null ? webView.getUrl() : "";
374     if(url.equals("")){
375         Log.d("WebViewActivityMenu", "null url");
376         Toast.makeText(this, "Url■■", Toast.LENGTH_SHORT).show();
377         return true;
378     }
379
380 // ■■■■■■■■■■■■■■■■■NetworkOnMainThreadException
381 ExecutorService executor = Executors.newSingleThreadExecutor();
382 Handler handler = new Handler(Looper.getMainLooper());
383
384 executor.execute(() -> {

```

```

385         try {
386             Log.d("WebViewActivityMenu", "@1.01");
387             String content = CrawlUtil.getArticleByUrl(url);
388             Log.d("WebViewActivityMenu", "@2");
389             // ■■■■■■■■■■UI
390             handler.post(() -> {
391                 try {
392                     // ■■■■■■■■■■
393                     ClipboardManager clipboard = (ClipboardManager) getS
394                     ClipData clip = ClipData.newPlainText("extracted_con
395                     clipboard.setPrimaryClip(clip);
396                     Log.d("WebViewActivityMenu", "@3");
397                     Toast.makeText(this, "■■Web■■■■■■■■■■■■■■■■■■■■", Toast.LENG
398                 } catch (Exception e) {
399                     Toast.makeText(this, "■■■■", Toast.LENGTH_SHORT).sho
400                     Log.d("WebViewActivityMenu", e.toString());
401                 }
402             });
403         } catch (Exception e) {
404             // ■■■■■■■■■■
405             handler.post(() -> {
406                 Toast.makeText(this, "■■■■", Toast.LENGTH_SHORT).show();
407                 Log.d("WebViewActivityMenu", e.toString());
408             });
409         }
410     });
411
412     return true;
413 } else if (item.getItemId() == R.id.action_toggle_navigation) {
414     toggleNavigationControls();
415     return true;
416 } else if (item.getItemId() == R.id.action_toggle_external_block) {
417     toggleExternalBlockMode();
418     return true;
419 } else if (item.getItemId() == R.id.action_add_tag) {
420     showAddTagDialog();
421     return true;
422 }
423 return super.onOptionsItemSelected(item);
424 }
425
426
427 private void setupWebView() {
428     attachControlRevealTouchListener();
429
430     // ■■ JavaScript
431     webView.getSettings().setJavaScriptEnabled(true);
432
433     // ■■■■■■
434     webView.getSettings().setBuiltInZoomControls(true);

```

```
435         webView.getSettings().setDisplayZoomControls(false);
436
437         // ■■ DOM storage
438         webView.getSettings().setDomStorageEnabled(true);
439
440         // ■■■■■■■■HTTP■HTTPS■
441         webView.getSettings().setMixedContentMode(WebSettings.MIXED_CONTENT_ALWA
442
443         // ■■■■■■■■
444         webView.getSettings().setMediaPlaybackRequiresUserGesture(false);
445
446         webView.setWebChromeClient(new WebChromeClient() {
447             @Override
448             public void onShowCustomView(View view, CustomViewCallback callback) {
449                 super.onShowCustomView(view, callback);
450             }
451
452             @Override
453             public void onPermissionRequest(PermissionRequest request) {
454                 runOnUiThread(() -> request.grant(request.getResources()));
455             }
456         });
457
458         // ■■JS■■■■■■■■■■Java■■■■■■■■■■
459         webView.addJavascriptInterface(new MediaInterfaceObject(), "AndroidMedia
460
461         webView.setWebViewClient(new WebViewClient() {
462             @Override
463             public boolean shouldOverrideUrlLoading(WebView view, String url) {
464                 Log.d("WVUrlTrace", "shouldOverrideUrlLoading(String) url=" + ur
465                 return handleUrlOverride(view, url, true);
466             }
467
468             @Override
469             public boolean shouldOverrideUrlLoading(WebView view, WebResourceReq
470                 String url = request != null && request.getUrl() != null
471                     ? request.getUrl().toString()
472                     : null;
473                 boolean isMainFrame = request == null || request.isForMainFrame(
474                 Log.d("WVUrlTrace", "shouldOverrideUrlLoading(Request) url=" + u
475                     + ", isMainFrame=" + isMainFrame);
476                 return handleUrlOverride(view, url, isMainFrame);
477             }
478
479             @Override
480             public void onPageFinished(WebView view, String url) {
481                 super.onPageFinished(view, url);
482                 try {
483                     String t = view.getTitle();
484                     if (t != null && !t.trim().isEmpty()) {
```

534

```
584         // http(s) ████████ WebView █████
```

```

585     if (url.startsWith("http://") || url.startsWith("https://")) {
586         Log.d("WVUrlTrace", "handleUrlOverride http(s) → load in WebView: "
587             + url);
588         return false;
589     }
590     // if (!isMainFrame) {
591     //     Log.d("WVUrlTrace", "handleUrlOverride early-return: url=" + url
592     //         + ", isMainFrame=" + isMainFrame);
593     //     return false;
594     // }
595
596     // 2=0=1=
597     int externalLinkMode = getExternalLinkMode();
598
599     // http(s) scheme snsdk://, intent://, weixin://
600     Log.d("WVUrlTrace", "handleUrlOverride non-http url=" + url
601         + ", mode=" + externalLinkMode);
602
603     if (externalLinkMode == 0) {
604         // 0
605         showExternalOpenConfirmDialog(url);
606     } else if (externalLinkMode == 1) {
607         // 1
608         openExternalUri(url);
609     }
610     // 2
611     return true;
612 }
613
614 private int getExternalLinkMode() {
615     try {
616         SharedPreferences prefs = getSharedPreferences("settings", MODE_PRIVATE);
617         return prefs.getInt("external_link_mode", 2); //
618     } catch (Exception ignored) {
619         return 2;
620     }
621 }
622
623 private void toggleExternalBlockMode() {
624     SharedPreferences prefs = getSharedPreferences("settings", MODE_PRIVATE);
625     int currentMode = prefs.getInt("external_link_mode", 2);
626     // 2(2) -> 0(2) -> 1(2) -> 2(2)
627     int newMode;
628     String message;
629     if (currentMode == 2) {
630         newMode = 0;
631         message = "2";
632     } else if (currentMode == 0) {
633         newMode = 1;
634         message = "0";
635     }
636     prefs.putInt("external_link_mode", newMode);
637     Log.d("WVUrlTrace", "toggleExternalBlockMode: " + message);
638 }

```

684

734

```

735             if (!trimmedTag.isEmpty()) {
736                 newTags.add(trimmedTag);
737             }
738         }
739
740         List<String> allTags = new ArrayList<>(existingTags);
741         allTags.addAll(newTags);
742
743         if (!allTags.isEmpty()) {
744             if (!newTags.isEmpty()) {
745                 RecentTagsManager.addRecentTags(this, newTags);
746             }
747             saveTagsToLink(url, allTags);
748         }
749     })
750     .setNegativeButton("■■■", null)
751     .show();
752 }
753
754 private void saveTagsToLink(String url, List<String> tags) {
755     new Thread(() -> {
756         LinkDao dao = new LinkDao(WebViewActivity.this);
757         try {
758             dao.open();
759             LinkItem item;
760
761             if (dao.urlExists(url)) {
762                 List<LinkItem> allLinks = dao.getAllLinks();
763                 item = null;
764                 for (LinkItem link : allLinks) {
765                     if (url.equals(link.getUrl())) {
766                         item = link;
767                         break;
768                     }
769                 }
770             } else {
771                 String title = pageTitleCache != null && !pageTitleCache.isEmpty
772                     ? pageTitleCache : "■■■■";
773                 item = new LinkItem(title, url, "", "", "");
774                 item.setClickCount(0);
775                 item.setSummary("");
776                 long id = dao.insertLink(item);
777                 item.setId(id);
778             }
779
780             if (item != null) {
781                 for (String tag : tags) {
782                     item.addTag(tag);
783                 }
784                 dao.updateLinkTags(item);

```

834

```

835     @Override
836     protected void onPause() {
837         isAppInForeground = false;
838         // ■■■■■■■■■■■■■■■■■■■■■■ AudioFocus■WebView ■■■■■■■■■■■■■■■■■■■■■■
839         Intent serviceIntent = new Intent(this, WebViewBackgroundService.class);
840         serviceIntent.putExtra("current_url", currentUrl);
841         startForegroundService(serviceIntent);
842         if (wakeLock != null && !wakeLock.isHeld()) {
843             wakeLock.acquire(30 * 60 * 1000L);
844         }
845         super.onPause();
846     }
847
848     private void initAudio() {
849         audioManager = (AudioManager) getSystemService(Context.AUDIO_SERVICE);
850
851         AudioAttributes audioAttributes = new AudioAttributes.Builder()
852             .setUsage(AudioAttributes.USAGE_MEDIA)
853             .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
854             .build();
855         audioFocusRequest = new AudioFocusRequest.Builder(AudioManager.AUDIOFOCUS_
856             .setAudioAttributes(audioAttributes)
857             .setWillPauseWhenDucked(false)
858             .setOnAudioFocusChangeListener(focusChange -> {
859                 if (focusChange == AudioManager.AUDIOFOCUS_LOSS
860                     || focusChange == AudioManager.AUDIOFOCUS_LOSS_TRANS
861                     if (webView != null) {
862                         webView.evaluateJavascript(
863                             "(function(){[].slice.call(document.getElementsB
864                             + ".concat([].slice.call(document.getElementsByT
865                             + ".forEach(function(e){e.pause();});})();)", null
866                         }
867                     } else if (focusChange == AudioManager.AUDIOFOCUS_GAIN) {
868                         if (audioPlaying && webView != null) {
869                             webView.evaluateJavascript(
870                                 "(function(){[].slice.call(document.getElementsB
871                                 + ".concat([].slice.call(document.getElementsByT
872                                 + ".forEach(function(e){e.play().catch(function(
873                             }
874                         }
875                     })
876                 .build();
877
878         PowerManager pm = (PowerManager) getSystemService(Context.POWER_SERVICE)
879         wakeLock = pm.newWakeLock(PowerManager.PARTIAL_WAKE_LOCK, "WebViewAudio:
880
881         mediaSession = new MediaSessionCompat(this, "WebViewAudio");
882         mediaSession.setFlags(MediaSessionCompat.FLAG_HANDLES_MEDIA_BUTTONS
883             | MediaSessionCompat.FLAG_HANDLES_TRANSPORT_CONTROLS);
884         mediaSession.setCallback(new MediaSessionCompat.Callback() {

```

第 93 页 / 共 112 页

1034


```
1085         if (!hasValidNavigationContext() || isNavigating) return;
1086         int previousIndex = (currentIndex - 1 + contextIds.length) % contextIds.
1087         loadByContextIndex(previousIndex);
1088     }
1089
1090     private void navigateToNext() {
1091         if (!hasValidNavigationContext() || isNavigating) return;
1092         int nextIndex = (currentIndex + 1) % contextIds.length;
1093         loadByContextIndex(nextIndex);
1094     }
1095
1096     private void loadByContextIndex(int index) {
1097         if (contextIds == null || index < 0 || index >= contextIds.length) retur
1098         if (isNavigating) return;
1099         isNavigating = true;
1100         long targetId = contextIds[index];
1101         new Thread(() -> {
1102             LinkDao dao = new LinkDao(this);
1103             String url = null;
1104             String title = null;
1105             try {
1106                 dao.open();
1107                 person.notfresh.readingshare.model.LinkItem item = dao.getLinkBy
1108                 if (item != null) {
1109                     url = item.getUrl();
1110                     title = item.getTitle();
1111                 }
1112             } catch (Exception e) {
1113                 Log.w("WebViewActivity", "loadByContextIndex lookup failed: " +
1114             } finally {
1115                 try { dao.close(); } catch (Exception ignored) {}
1116             }
1117             final String finalUrl = url;
1118             final String finalTitle = title;
1119             runOnUiThread(() -> {
1120                 if (finalUrl != null && webView != null) {
1121                     if (!finalUrl.equals(currentUrl)) {
1122                         webView.loadUrl(finalUrl);
1123                         currentUrl = finalUrl;
1124                         if (finalTitle != null && toolbar != null) toolbar.setTi
1125                     }
1126                     currentIndex = index;
1127                     showControlsTemporarily();
1128                 } else {
1129                     Toast.makeText(this, "■■■■■■■■", Toast.LENGTH_SHORT).show();
1130                 }
1131                 isNavigating = false;
1132             });
1133         }).start();
1134     }
```


83 }

```
1  package person.notfresh.readingshare;
2
3  import android.database.Cursor;
4  import android.os.Bundle;
5  import androidx.appcompat.app.AppCompatActivity;
6  import androidx.recyclerview.widget.LinearLayoutManager;
7  import androidx.recyclerview.widget.RecyclerView;
8  import com.google.android.material.tabs.TabLayout;
9  import person.notfresh.readingshare.adapter.ClickStatisticsAdapter;
10 import person.notfresh.readingshare.db.LinkDao;
11
12 public class ClickStatisticsActivity extends AppCompatActivity {
13
14     private LinkDao linkDao;
15     private RecyclerView recyclerView;
16     private ClickStatisticsAdapter adapter;
17     private TabLayout tabLayout;
18
19     @Override
20     protected void onCreate(Bundle savedInstanceState) {
21         super.onCreate(savedInstanceState);
22         setContentView(R.layout.activity_click_statistics);
23
24         linkDao = new LinkDao(this);
25         linkDao.open();
26
27         recyclerView = findViewById(R.id.recyclerView);
28         recyclerView.setLayoutManager(new LinearLayoutManager(this));
29
30         tabLayout = findViewById(R.id.tabLayout);
31         tabLayout.addTab(tabLayout.newTab().setText("■■■■"));
32         tabLayout.addTab(tabLayout.newTab().setText("■■■■"));
33
34         loadData("month");
35
36         tabLayout.addOnTabSelectedListener(new TabLayout.OnTabSelectedListener()
37             @Override
38             public void onTabSelected(TabLayout.Tab tab) {
39                 String period = tab.getPosition() == 0 ? "month" : "week";
40                 loadData(period);
41             }
42
43             @Override
44             public void onTabUnselected(TabLayout.Tab tab) {}
45
46             @Override
47             public void onTabReselected(TabLayout.Tab tab) {}
48         });
49     }
50 }
```

```
51     private void loadData(String period) {
52         Cursor cursor = linkDao.getClickStatistics(period);
53         adapter = new ClickStatisticsAdapter(cursor);
54         recyclerView.setAdapter(adapter);
55     }
56
57     @Override
58     protected void onDestroy() {
59         super.onDestroy();
60         linkDao.close();
61     }
62 }
1  package person.notfresh.readingshare;
2
3  import android.Manifest;
4  import android.content.BroadcastReceiver;
5  import android.content.Context;
6  import android.content.Intent;
7  import android.content.IntentFilter;
8  import android.content.SharedPreferences;
9  import android.content.pm.PackageManager;
10 import android.graphics.Bitmap;
11 import android.net.Uri;
12 import android.os.Bundle;
13 import android.provider.MediaStore;
14 import android.text.TextUtils;
15 import android.util.Log;
16 import android.view.MenuItem;
17 import android.widget.Button;
18 import android.widget.ImageView;
19 import android.widget.Toast;
20
21 import androidx.annotation.NonNull;
22 import androidx.appcompat.app.AlertDialog;
23 import androidx.appcompat.app.AppCompatActivity;
24 import androidx.appcompat.widget.Toolbar;
25 import androidx.core.app.ActivityCompat;
26 import androidx.core.content.ContextCompat;
27 import androidx.core.content.FileProvider;
28 import androidx.recyclerview.widget.LinearLayoutManager;
29 import androidx.recyclerview.widget.RecyclerView;
30
31 import com.google.android.material.textfield.TextInputEditText;
32 import java.io.File;
33 import java.io.FileOutputStream;
34 import java.io.IOException;
35 import org.json.JSONArray;
36 import org.json.JSONException;
37
38 import person.notfresh.readingshare.adapter.PublishedTagsAdapter;
```

```
39
40 public class UserProfileActivity extends AppCompatActivity {
41     private static final int REQUEST_IMAGE_CAPTURE = 1;
42     private static final int REQUEST_GALLERY = 2;
43     private static final int PERMISSION_REQUEST_CODE = 100;
44
45     private ImageView profileImage;
46     private TextInputEditText usernameEdit;
47     private TextInputEditText emailEdit;
48     private Uri currentPhotoUri;
49     private RecyclerView publishedTagsRecycler;
50     private PublishedTagsAdapter publishedTagsAdapter;
51     private BroadcastReceiver updateReceiver;
52
53     @Override
54     protected void onCreate(Bundle savedInstanceState) {
55         try {
56             Log.d("UserProfileActivity", "■■■■■■■■");
57             super.onCreate(savedInstanceState);
58             setContentView(R.layout.activity_user_profile);
59
60             // ■■ Toolbar
61             Log.d("UserProfileActivity", "■■■■■Toolbar");
62             Toolbar toolbar = findViewById(R.id.toolbar);
63             setSupportActionBar(toolbar);
64             getSupportActionBar().setDisplayHomeAsUpEnabled(true);
65             getSupportActionBar().setTitle("■■■■■");
66
67             // ■■■■■■
68             Log.d("UserProfileActivity", "■■■■■■■■■■");
69             profileImage = findViewById(R.id.profile_image);
70             usernameEdit = findViewById(R.id.edit_username);
71             emailEdit = findViewById(R.id.edit_email);
72             Button saveButton = findViewById(R.id.btn_save);
73             ImageView editAvatar = findViewById(R.id.edit_avatar);
74
75             // ■■■■■■■■■■
76             Log.d("UserProfileActivity", "■■■■■■■■■■■");
77             loadUserProfile();
78
79             // ■■■■■■■■■■■■
80             editAvatar.setOnClickListener(v -> {
81                 Log.d("UserProfileActivity", "■■■■■■■■■■■");
82                 showImagePickerDialog();
83             });
84
85             // ■■■■■■■■■■■■
86             saveButton.setOnClickListener(v -> {
87                 Log.d("UserProfileActivity", "■■■■■■■■■■");
88                 saveUserProfile();
```

```

89         });
90
91         // ■■■■ RecyclerView
92         Log.d("UserProfileActivity", "■■■■■RecyclerView");
93         publishedTagsRecycler = findViewById(R.id.published_tags_recycler);
94         publishedTagsRecycler.setLayoutManager(new LinearLayoutManager(this));
95         publishedTagsAdapter = new PublishedTagsAdapter();
96         publishedTagsRecycler.setAdapter(publishedTagsAdapter);
97
98         // ■■■■■■■■
99         Log.d("UserProfileActivity", "■■■■■■■■■■");
100        updateReceiver = new BroadcastReceiver() {
101            @Override
102            public void onReceive(Context context, Intent intent) {
103                Log.d("UserProfileActivity", "■■■■■■■");
104                loadPublishedTags();
105            }
106        };
107        registerReceiver(updateReceiver, new IntentFilter("UPDATE_PUBLISHED_
108
109        loadPublishedTags();
110        Log.d("UserProfileActivity", "■■■■■■■");
111    } catch (Exception e) {
112        Log.e("UserProfileActivity", "onCreate■■■■■", e);
113    }
114 }
115
116 private void showImagePickerDialog() {
117     String[] options = {"■■■", "■■■■■■■"};
118     new AlertDialog.Builder(this)
119         .setTitle("■■■■■")
120         .setItems(options, (dialog, which) -> {
121             if (which == 0) {
122                 checkCameraPermissionAndLaunch();
123             } else {
124                 openGallery();
125             }
126         })
127         .show();
128 }
129
130 private void checkCameraPermissionAndLaunch() {
131     if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
132         != PackageManager.PERMISSION_GRANTED) {
133         ActivityCompat.requestPermissions(this,
134             new String[]{Manifest.permission.CAMERA},
135             PERMISSION_REQUEST_CODE);
136     } else {
137         launchCamera();
138     }

```

```

139     }
140
141     private void launchCamera() {
142         Intent takePictureIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
143         if (takePictureIntent.resolveActivity(getPackageManager()) != null) {
144             File photoFile = createImageFile();
145             if (photoFile != null) {
146                 currentPhotoUri = FileProvider.getUriForFile(this,
147                     getPackageName() + ".provider",
148                     photoFile);
149                 takePictureIntent.putExtra(MediaStore.EXTRA_OUTPUT, currentPhotoUri);
150                 startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE);
151             }
152         }
153     }
154
155     private void openGallery() {
156         // TODO: ■■■■■■ ImageUtil.createGalleryPickerIntent() ■■■■■■
157         Intent intent = new Intent(Intent.ACTION_PICK,
158             MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
159         startActivityForResult(intent, REQUEST_GALLERY);
160     }
161
162     private File createImageFile() {
163         File storageDir = getExternalFilesDir("profile_images");
164         File image = new File(storageDir, "profile_image.jpg");
165         try {
166             if (!image.exists()) {
167                 image.createNewFile();
168             }
169             return image;
170         } catch (IOException e) {
171             e.printStackTrace();
172             return null;
173         }
174     }
175
176     @Override
177     protected void onActivityResult(int requestCode, int resultCode, Intent data) {
178         super.onActivityResult(requestCode, resultCode, data);
179         if (resultCode == RESULT_OK) {
180             try {
181                 Bitmap bitmap = null;
182                 if (requestCode == REQUEST_IMAGE_CAPTURE) {
183                     if (currentPhotoUri != null) {
184                         bitmap = MediaStore.Images.Media.getBitmap(getContentResolver, currentPhotoUri);
185                     }
186                 } else if (requestCode == REQUEST_GALLERY && data != null) {
187                     Uri selectedImage = data.getData();
188                     bitmap = MediaStore.Images.Media.getBitmap(getContentResolver, selectedImage);
189                 }
190             } catch (IOException e) {
191                 e.printStackTrace();
192             }
193         }
194     }

```

```

189         }
190
191         if (bitmap != null) {
192             // ■■■■
193             // TODO: ■■■■■■ ImageUtil.resizeBitmap() ■■■■
194             bitmap = getResizedBitmap(bitmap, 500); // ■■■■■■ 500px
195
196             // ■■■■■■■■
197             File filesDir = getFilesDir();
198             File imageFile = new File(filesDir, "profile_" + System.curr
199
200             FileOutputStream fos = new FileOutputStream(imageFile);
201             bitmap.compress(Bitmap.CompressFormat.JPEG, 90, fos);
202             fos.close();
203
204             // ■■ UI ■■■■ URI
205             profileImage.setImageBitmap(bitmap);
206             saveImageUri(imageFile.getAbsolutePath());
207         }
208     } catch (Exception e) {
209         Log.e("UserProfile", "Error processing image", e);
210         Toast.makeText(this, "■■■■■■■■", Toast.LENGTH_SHORT).show();
211     }
212 }
213 }
214
215 // TODO: ■■■■■■ ImageUtil.resizeBitmap() ■■■■
216 private Bitmap getResizedBitmap(Bitmap image, int maxSize) {
217     int width = image.getWidth();
218     int height = image.getHeight();
219
220     float bitmapRatio = (float) width / (float) height;
221     if (bitmapRatio > 1) {
222         width = maxSize;
223         height = (int) (width / bitmapRatio);
224     } else {
225         height = maxSize;
226         width = (int) (height * bitmapRatio);
227     }
228
229     return Bitmap.createScaledBitmap(image, width, height, true);
230 }
231
232 private void loadUserProfile() {
233     try {
234         Log.d("UserProfileActivity", "■■■■■■■■");
235         SharedPreferences prefs = getSharedPreferences("UserProfile", MODE_P
236         String username = prefs.getString("username", "");
237         String email = prefs.getString("email", "");
238         String imageUri = prefs.getString("profile_image", "");

```



```
289         editor.putString("profile_image", uri);
290         editor.apply();
291     }
292
293     @Override
294     public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
295         super.onRequestPermissionsResult(requestCode, permissions, grantResults);
296         if (requestCode == PERMISSION_REQUEST_CODE) {
297             if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
298                 launchCamera();
299             } else {
300                 Toast.makeText(this, "■■■■■■■■■■", Toast.LENGTH_SHORT).show();
301             }
302         }
303     }
304 }
305
306 @Override
307 public boolean onOptionsItemSelected(MenuItem item) {
308     if (item.getItemId() == android.R.id.home) {
309         onBackPressed();
310         return true;
311     }
312     return super.onOptionsItemSelected(item);
313 }
314
315 private void loadPublishedTags() {
316     try {
317         Log.d("UserProfileActivity", "■■■■■■■■■■");
318         SharedPreferences prefs = getSharedPreferences("UserProfile", MODE_PRIVATE);
319         String publishedTagsStr = prefs.getString("published_tags", "");
320         Log.d("UserProfileActivity", "■■■■■■■■■■: " + publishedTagsStr);
321
322         JSONArray publishedTags = new JSONArray(publishedTagsStr);
323         if (publishedTagsAdapter != null) {
324             publishedTagsAdapter.updateData(publishedTags);
325             Log.d("UserProfileActivity", "■■■■■■■■■■");
326         } else {
327             Log.e("UserProfileActivity", "■■■■null");
328         }
329     } catch (Exception e) {
330         Log.e("UserProfileActivity", "loadPublishedTags■■■■", e);
331         if (publishedTagsAdapter != null) {
332             publishedTagsAdapter.updateData(new JSONArray());
333         }
334     }
335 }
336
337 @Override
338 protected void onResume() {
```

```

339         super.onResume();
340         loadPublishedTags(); // ██████████
341     }
342
343     @Override
344     protected void onDestroy() {
345         try {
346             Log.d("UserProfileActivity", "██████");
347             super.onDestroy();
348             if (updateReceiver != null) {
349                 unregisterReceiver(updateReceiver);
350                 Log.d("UserProfileActivity", "██████████");
351             }
352         } catch (Exception e) {
353             Log.e("UserProfileActivity", "onDestroy████", e);
354         }
355     }
356 }
1  package person.notfresh.readingshare;
2
3  import android.app.Notification;
4  import android.app.NotificationChannel;
5  import android.app.NotificationManager;
6  import android.app.PendingIntent;
7  import android.app.Service;
8  import android.content.Context;
9  import android.content.Intent;
10 import android.os.Build;
11 import android.os.IBinder;
12 import android.os.PowerManager;
13 import android.util.Log;
14
15 import androidx.core.app.NotificationCompat;
16
17 public class WebViewBackgroundService extends Service {
18     private static final String TAG = "WebViewBgService";
19     private static final int NOTIFICATION_ID = 1;
20     private static final String CHANNEL_ID = "WebViewAudioChannel";
21     public static final String ACTION_PLAY_PAUSE = "person.notfresh.readingshare
22     public static final String ACTION_STOP = "person.notfresh.readingshare.STOP"
23
24     private PowerManager.WakeLock serviceLock;
25     private String currentUrl;
26     private boolean isPaused = false;
27
28     // ██████████ Activity ██████████ IPC
29     public interface MediaCallback {
30         void onPlayRequested();
31         void onPauseRequested();
32         void onStopRequested();

```

```
33     }
34     private static MediaCallback sMediaCallback;
35     public static void setMediaCallback(MediaCallback callback) {
36         sMediaCallback = callback;
37     }
38
39     @Override
40     public void onCreate() {
41         super.onCreate();
42         Log.d(TAG, "■■■■■■");
43
44         // ■■■■■■WakeLock■■■■CPU■■■
45         PowerManager pm = (PowerManager) getSystemService(Context.POWER_SERVICE);
46         serviceLock = pm.newWakeLock(PowerManager.PARTIAL_WAKE_LOCK,
47             "WebViewBackgroundService::ServiceLock");
48         serviceLock.acquire(30 * 60 * 1000L);
49     }
50
51     @Override
52     public int onStartCommand(Intent intent, int flags, int startId) {
53         Log.d(TAG, "■■■■■■");
54         createNotificationChannel();
55
56         // ■■■■■■■■ action
57         if (intent != null) {
58             String action = intent.getAction();
59             if (ACTION_PLAY_PAUSE.equals(action)) {
60                 isPaused = !isPaused;
61                 Log.d(TAG, "Play/Pause ■■■■, isPaused=" + isPaused + ", callback");
62                 if (sMediaCallback != null) {
63                     if (isPaused) {
64                         sMediaCallback.onPauseRequested();
65                     } else {
66                         sMediaCallback.onPlayRequested();
67                     }
68                 }
69                 // ■■■■
70                 startForeground(NOTIFICATION_ID, buildNotification());
71                 return START_STICKY;
72             } else if (ACTION_STOP.equals(action)) {
73                 Log.d(TAG, "Stop ■■■■, callback=" + (sMediaCallback != null));
74                 if (sMediaCallback != null) {
75                     sMediaCallback.onStopRequested();
76                 }
77                 stopForeground(true);
78                 stopSelf();
79                 return START_NOT_STICKY;
80             }
81         }
82     }
```

```

83         // ■■■■
84         if (intent != null) {
85             String url = intent.getStringExtra("current_url");
86             if (url != null) currentUrl = url;
87             isPaused = false;
88         }
89
90         startForeground(NOTIFICATION_ID, buildNotification());
91         return START_STICKY;
92     }
93
94     private Notification buildNotification() {
95         // ■■■■■■■■
96         Intent notificationIntent = new Intent(this, WebViewActivity.class);
97         if (currentUrl != null && !currentUrl.isEmpty()) {
98             notificationIntent.putExtra("url", currentUrl);
99         }
100         PendingIntent contentIntent = PendingIntent.getActivity(
101             this, 0, notificationIntent,
102             PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
103
104         // ■■/■■■■
105         Intent playPauseIntent = new Intent(this, WebViewBackgroundService.class);
106         playPauseIntent.setAction(ACTION_PLAY_PAUSE);
107         PendingIntent playPausePending = PendingIntent.getService(
108             this, 1, playPauseIntent,
109             PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
110
111         // ■■■■
112         Intent stopIntent = new Intent(this, WebViewBackgroundService.class);
113         stopIntent.setAction(ACTION_STOP);
114         PendingIntent stopPending = PendingIntent.getService(
115             this, 2, stopIntent,
116             PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
117
118         int playPauseIcon = isPaused ? R.drawable.ic_play : R.drawable.ic_pause;
119         String playPauseLabel = isPaused ? "■" : "■";
120         String statusText = isPaused ? "■■■" : "■■■■";
121
122         return new NotificationCompat.Builder(this, CHANNEL_ID)
123             .setContentTitle("■■■■■")
124             .setContentText(statusText + " - ■■■■■")
125             .setSmallIcon(R.mipmap.ic_launcher)
126             .setContentIntent(contentIntent)
127             .setOngoing(true)
128             .setPriority(NotificationCompat.PRIORITY_HIGH)
129             .addAction(playPauseIcon, playPauseLabel, playPausePending)
130             .addAction(R.drawable.ic_close, "■", stopPending)
131             .setStyle(new androidx.media.app.NotificationCompat.MediaStyle()
132                 .setShowActionsInCompactView(0, 1))

```

```

133         .build();
134     }
135
136     private void createNotificationChannel() {
137         // Android 8.0
138         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
139             NotificationChannel channel = new NotificationChannel(
140                 CHANNEL_ID,
141                 "WebView",
142                 NotificationManager.IMPORTANCE_LOW
143             );
144             channel.setDescription("WebView");
145
146             NotificationManager notificationManager = getSystemService(NotificationManager.class);
147             notificationManager.createNotificationChannel(channel);
148         }
149     }
150
151     @Override
152     public IBinder onBind(Intent intent) {
153         return null;
154     }
155
156     @Override
157     public void onDestroy() {
158         if (serviceLock != null && serviceLock.isHeld()) {
159             serviceLock.release();
160         }
161         super.onDestroy();
162         Log.d(TAG, "");
163     }
164 }
1
package person.notfresh.readingshare.config;
2
3 /**
4  *
5  *
6  */
7 public class Config {
8
9     // API
10    public static class API {
11        // AI API
12        public static final String DUXIANG_API_URL = "https://duxiang.ai/api/abs";
13        public static final String DUXIANG_API_KEY = "notfresh@duxiang.ai";
14    }
15
16    //
17    public static class Network {
18        //

```

54 }